Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Manouba

Institut supérieur des Arts Multimédias



Mémoire de fin d'études

Préparé en vue de l'obtention du diplôme de Licence en Micro-Informatique et Machines Embarquées

Développement d'Un système embarqué de surveillance du trafic routier en temps réel

Réalisé par

Touka JUINI

Encadré par

Mr. Amine KCHICHE

Année Universitaire : 2024 – 2025

Dédicaces

À ma précieuse mère, merci pour tout ce que tu fais pour nous. Ton dévouement, ton soutien inébranlable, et ta patience, même dans les moments difficiles, m'ont toujours profondément touché. Tu ne te plains jamais, tu es toujours présente pour nous, pour nos études, avec tout ton amour. Cela signifie énormément pour moi.

À mon tendre papa, notre soutien indéfectible et notre source d'inspiration. Merci pour tes sacrifices sans fin, pour toujours vouloir que nous progressions et devenions meilleurs. Ton amour et ton soutien nous poussent chaque jour.

À ma sœur bien-aimée, ma complice de chaque instant, ma source inépuisable de force et de bonheur. Notre lien est précieux ; c'est toujours toi qui m'apportes réconfort et amour quand j'en ai besoin.

À mes tantes, oncles, cousins, cousines et grands-pères, merci d'avoir toujours été présents pour moi depuis le début de mon parcours. Votre soutien, vos encouragements et votre affection ont été précieux et essentiels pour moi.

Remerciement

Avant tout, je rends grâce à Dieu, car c'est par Sa volonté souveraine et Sa bienveillance infinie que j'ai pu parvenir jusqu'à ce stade de mon parcours.

Je tiens à exprimer ma profonde reconnaissance à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail. Leur soutien constant, leurs encouragements précieux et leur aide bienveillante ont été des piliers essentiels tout au long de cette aventure.

Je remercie tout particulièrement mon encadrant à l'ISAMM ,**Monsieur Amine Kchiche**, pour son accompagnement attentif et chaleureux. Ses conseils avisés, ses remarques constructives et sa disponibilité sans faille ont grandement facilité ma progression et orienté mes efforts dans la bonne direction.

Enfin, j'adresse mes sincères remerciements aux membres du jury, pour avoir consacré leur temps avec sérieux et attention à l'évaluation de ce travail, contribuant ainsi à enrichir sa qualité et sa rigueur.

Table des matières

Introduction générale **1**

1 Contexte du projet **3**
1.1 Introduction . . . 3
1.2 Présentation de l'organisme d'accueil . . . 3
1.2.1 Présentation de l'ISAMM : . . . 3
1.2.2 La spécialité MIME . . . 4
1.3 Présentation du projet . . . 4
1.3.1 Cadre de projet . . . 4
1.3.2 Problématique . . . 4
1.4 Etude de l'existant . . . 5
1.4.1 Description de l'existant . . . 5
1.4.2 Critique de l'existant . . . 6
1.4.3 Solution proposée . . . 7
1.5 Méthodologie de travail et planification . . . 8
1.5.1 Méthodologie de gestion de projet . . . 8
1.5.2 Méthodologie de conception . . . 9
1.6 Conclusion . . . 10

2 Analyse des besoins **11**
2.1 Introduction . . . 11
2.2 Analyse des besoins . . . 11
2.2.1 Identification des acteurs . . . 11
2.2.2 Besoins fonctionnels . . . 11
2.2.3 Besoins non fonctionnels . . . 12
2.3 Spécification des besoins . . . 12
2.3.1 Cas d'utilisation global . . . 12
2.3.2 les cas d'utilisation détaillées . . . 13
2.4 Conclusion . . . 16

3 Modélisation **17**
3.1 Introduction . . . 17
3.2 Structure globale du système . . . 17
3.2.1 Composants de l'architecture . . . 17
3.2.2 Diagramme de déploiement . . . 19
3.2.3 Protocoles et interfaces de communication . . . 19
3.3 Modélisation dynamique du système . . . 20
3.3.1 Diagramme d'activité global . . . 20
3.3.2 Diagramme d'activité « Connectivité WiFi et MQTT » . . . 22

3.3.3 Diagramme d'activité « Detection de vehicule » 24
3.4 Diagrammes de séquence 26
3.4.1 Diagramme de séquence « Détection de véhicule » 26
3.4.2 Diagramme de séquence « Réception et traitement des données » . 28
3.4.3 Diagramme de séquence « Filtrage des Doublons » 29
3.4.4 Diagramme de séquence « Affichage sur le tableau de bord » 29
3.4.5 Diagramme de séquence « Configuration des routes » 31
3.4.6 Étude d'une solution alternative basée sur l'interruption 31
3.5 Base de Données 33
3.6 Conclusion 33

4 Réalisation **34**
4.1 Introduction 34
4.2 Environnement logiciel 34
4.2.1 Partie embarqué 34
4.2.2 Partie front-end 35
4.2.3 Partie Backend 36
4.2.4 Bibliothèques utilisées : 38
4.2.5 Outils de modélisation des diagrammes et de rédaction 38
4.3 Ressources matérielles 40
4.3.1 Noeud de capteur 40
4.3.2 Circuit d'alimentation 43
4.3.3 Circuit de recharge 45
4.4 Circuit électrique 47
4.5 Architecture du système 47
4.6 Implémentation et interfaces 48
4.6.1 Configuration du capteur VL53L0X[16] 48
4.6.2 Test et résultats 50
4.6.3 Réalisation du système de détection 54
4.7 Conclusion 56

Conclusion Générale **57**

Bibliographie **58**

Table des figures

1.1 Logo de société 4
1.2 Smartmicro UMRR-96 5
1.3 FLIR TrafiCam x-stream™ 6
1.4 Sensys Networks VDS240 6
1.5 Disposition des capteurs en bord de route 8
1.6 Représentation du modèle en spirale 9
1.7 Logo d'UML 10
2.8 Diagramme de cas d'utilisation global 13
2.9 Diagramme de cas d'utilisation « Superviser la détection des véhicules » . 14
2.10 Diagramme de cas d'utilisation «Visualiser les données» 15
2.11 Diagramme de cas d'utilisation «Gérer les routes et les noeuds» 16
3.12 Diagramme de composants du système 18
3.13 Diagramme de déploiement global 19
3.14 Diagramme de fonctionnement du protocole MQTT 20
3.15 Diagramme d'activité global 21
3.16 Diagramme d'activité « Connectivité WiFi et MQTT » 23
3.17 Diagramme d'activité « Detection des vehicules » 25
3.18 Diagramme de séquence «Détection de véhicule » 27
3.19 Diagramme de séquence « Detection de vehicule » en mode interruption . . 28
3.20 Diagramme de séquence « Réception et traitement des données » 29
3.21 Diagramme de séquence « Filtrage des Doublons » 29
3.22 Diagramme de séquence « Affichage sur le tableau de bord » 30
3.23 Diagramme de séquence « Configuration des nœuds et des routes » 31
3.24 Diagramme d'activité globale en mode interruption 32
3.25 Schéma Entity-Relationship (ER) de la base de données 33
4.26 Logo arduino ide 34
4.27 Logo VS 35
4.28 Logo html 35
4.29 Logo JS 36
4.30 Logo CSS 36
4.31 Logo Flask 37
4.32 Logo python 37
4.33 Logo WebSocket 38
4.34 Logo Overleaf 39
4.35 Logo Draw io 39
4.36 Logo EasyEDA 40
4.37 ESP 32 41
4.38 Principe de fonctionnement de « Vl53l0X » 42
4.39 VL53L0X 42

4.40 Principe de fonctionnement du capteur d'ultrasons 43
4.41 Capteur ultrason HY-SRF05 43
4.42 Batterie Li-ion 2500mAh 3.7V 44
4.43 Module de charge 45
4.44 Panneau Solaire 4W/6V 46
4.45 Module LM2596 (Convertisseur DC-DC) 46
4.46 Schéma électrique du circuit 47
4.47 Le schéma du système 48
4.48 Résultat de calibrage 51
4.49 Résultat du test orienté vers la fiabilité 52
4.50 Résultat du test orienté vers la rapidité 54
4.51 Interface de visualisation des données graphiques 55
4.52 Prototype final réel 56

Liste des tableaux

1.1 Tableau comparatif de l'existant 7
2.2 Description textuelle du cas d'utilisation «Superviser la detection des véhicules» 14
2.3 Description textuelle du cas d'utilisation «Visualiser les données» 15
2.4 Description textuelle du cas d'utilisation «Gérer les routes et les noeuds» . 16

Introduction générale

Le trafic est un élément important dans l'étude des infrastructures et de l'urbanisme. À mesure que les villes deviennent plus encombrées et que le nombre de véhicules sur les routes augmente, les zones urbaines deviennent plus peuplées, ce qui affecte négativement la circulation et augmente les temps de trajet. Selon les dernières statistiques, le temps perdu dans les embouteillages est d'environ 120 heures[1] par an, et cela nuit également à l'économie, coûtant environ 100 à 150 milliards d'euros par an [2] dans l'Union européenne.

Il est donc devenu important d'utiliser des systèmes et des technologies qui nous permettent de collecter et analyser les données de trafic en temps réel pour améliorer les décisions en matière de trafic et obtenir une vue d'ensemble du flux et de la densité du trafic.

Dans ce contexte, ce projet repose sur l'utilisation de l'Internet des objets et de capteurs installés sur la route qui détectent les véhicules et leur passage , puis, via un protocole de communication, transmettent les informations à un centre de traitement central pour extraire une image fidéle de la situation du trafic, qui est ensuite affichée sur un tableau de bord interactif.

Les objectifs de ce projet sont d'identifier la densité du trafic afin de fournir des prévisions futures des niveaux de trafic et de la couverture du nombre de véhicules sur la route.
Le but de ce rapport est de présenter en détail la manière dont ce projet a été conçu et réalisé.

Le projet est divisé en quatre chapitres qui reflètent l'avancement des travaux sur le projet :

Dans le premier chapitre, nous présenterons l'organisme d'accueil, le contexte du projet, ainsi que les solutions existantes et la méthodologie de travail.

Dans le deuxième chapitre, nous identifierons les principaux acteurs du système et analyserons en détail leurs besoins, lesquels seront représentés à travers des diagrammes de cas d'utilisation.

Le troisième chapitre est consacré à la conception du système. Il présente les principaux choix architecturaux à travers des diagrammes de séquence et d'activité, illustrant le déroulement des interactions entre les différents composants du système.

Le quatrième chapitre porte sur la réalisation complète du prototype final. Il met en

évidence les choix logiciels et technologiques adoptés, tout en détaillant les étapes de mise en œuvre concrète du système.

chapitre 1

Contexte du projet

1.1 Introduction

Dans ce premier chapitre, nous commencerons par définir le contexte et le cadre général du projet. Nous définirons également l'organisme d'accueil au sein duquel le projet a été mis en œuvre. Après cela, nous réaliserons une étude dans laquelle nous discuterons du problème et analyserons les solutions disponibles sur le marché et leurs inconvénients afin de présenter ensuite la solution proposée. Nous évoquerons également la méthode de travail suivie pour réaliser ce projet.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation de l'ISAMM :

L'Institut supérieur des arts multimédia de La Manouba (ISAMM) est un établissement public tunisien affilié au campus universitaire de la Manouba. Elle a été créée le 30 mai 2000. Les objectifs de l'institution visent à fournir une éducation et une formation appropriées aux professions futures et aux exigences mondiales, et d'établir plusieurs liens avec la communauté et le secteur professionnel à travers des clubs et des forums d'entreprises. Elle contient également de la diversité, car l'institution comprend environ 14 spécialisations, y compris des cours de master et d'ingénierie, nous proposons certaines de ces spécialisations telles que :

1.2.1 Présentation de l'ISAMM

L'Institut Supérieur des Arts Multimédias de la Manouba (ISAMM) propose plusieurs filières spécialisées dans les domaines technologiques et artistiques, notamment :

— Le micro-informatique et les machines embarquées MIME.
— Les BIG DATA et l'analyse des données.
— Le développement de jeux vidéo.
— La multimédia.
— Le cinéma.
— L'audiovisuel.



FIGURE 1.1 – Logo de société

1.2.2 La spécialité MIME

La spécialisation MIME (Micro-informatique et Machines Embarquées) est une licence appliquée au sein du programme de l'université qui vise à former des individus compétents et polyvalents dans le domaine des machines embarquées et de l'Internet des objets. Il vise à doter les étudiants des compétences fondamentales pour répondre aux besoins du marché international. Ces compétences comprennent le développement de expertises en programmation en temps réel, ainsi que la capacité à gérer des équipements et des technologies tels que des nœuds de capteurs et des réseaux, ainsi que la capacité à utiliser divers protocoles de communication et à les intégrer dans des systèmes complexes.

1.3 Présentation du projet

1.3.1 Cadre de projet

La conception et le développement d'un « Système de Surveillance du trafic en temps réel » s'inscrivent dans le cadre du projet de fin d'études en vue de l'obtention de diplôme de licence en science de l'informatique, spécialisée en Micro-Informatique et Machines Embarquées.

1.3.2 Problématique

La régulation du trafic est actuellement l'un des enjeux les plus importants liés à la mobilité urbaine, car elle impacte significativement la qualité de la mobilité des citoyens et leur bien-être en termes de gestion du temps. Il est donc important de fournir une image claire du flux de trafic sur la route et de déterminer sa densité, ce qui permet d'intégrer des solutions claires et adaptatives pour améliorer la fluidité du trafic et réduire les pertes de temps ou les accidents résultant de la densité du trafic. De nombreuses technologies peuvent apporter une solution à ces problèmes, mais elles peuvent également poser d'autres problèmes en termes de précision, de coût élevé d'installation et de maintenance continue, ou de difficulté de leur intégration dans l'infrastructure. Il est donc nécessaire de rechercher une solution efficace, facile à gérer et à exploiter, et abordable.

1.4 Etude de l'existant

1.4.1 Description de l'existant

Dans le but d'améliorer la surveillance du trafic, plusieurs solutions ont vu le jour afin d'assurer une couverture et un suivi en temps réel des différents aspects du trafic et de l'état des routes. Ces solutions reposent sur diverses technologies et capteurs, tels que les caméras et les signaux mobiles, tout en intégrant des méthodes de surveillance à travers des tableaux de bord ou des applications. L'objectif est de mettre en place des systèmes autonomes et réactifs. Nous présenterons un ensemble de ces solutions, accompagné d'une explication simplifiée pour chacune :

a- Smartmicro UMRR-96

Le Smartmicro UMRR-96 [3] est une solution de la société allemande Smartmicro. Il est basé sur un capteur radar Doppler qui calcule la vitesse et la distance en transmettant des ondes radio. Il peut être connecté via Ethernet ou des réseaux mobiles et est souvent monté sur des poteaux ou des mâts. Il nécessite un réglage précis de l'angle pour pouvoir couvrir correctement la route. Il dispose d'un tableau de bord de suivi et est actuellement utilisé à Singapour et à Singapour pour détecter les véhicules aux intersections. Cependant, le capteur reste sensible aux interférences électromagnétiques.



Figure 1.2 – Smartmicro UMRR-96

b- FLIR TrafiCam x-stream™

Le FLIR TrafiCam x-stream™ [4] est une caméra intelligente développée par l'entreprise américaine spécialisée dans les technologies de détection thermique et les solutions de surveillance "Teledyne FLIR" . Elle utilise une caméra haute définition et la vision par ordinateur embarquée pour détecter les véhicules, calculer les files d'attente et analyser le trafic en temps réel. Elle remplace les boucles magnétiques traditionnelles, offrant une détection non intrusive et fonctionne sans serveur externe. Elle est compatible avec des protocoles comme NTCIP et TCP/IP .
Cependant, son coût est relativement élevé et elle est principalement utilisée dans les grandes agglomérations pour gérer efficacement la circulation, comme à Paris (France).



FIGURE 1.3 – FLIR TrafiCam x-stream™

c- Sensys Networks VDS240 [5]

Cette solution, fournie par Sensys Networks, combine les avantages de la technologie des capteurs radar 24 GHz et des boucles magnétiques qui sont affectés par les métaux passant à proximité, provoquant des perturbations qui génèrent des signaux calculés pour obtenir la densité des véhicules. Le radar est installé sur des poteaux et est connecté via des réseaux mobiles, tandis que les boucles sont placées sous terre et sont connectées via un câble RS-485. Il fournit également un tableau de bord SensysNet pour surveiller le trafic. Cette solution est mise en œuvre à San Francisco , et son coût est généralement de l'ordre de 5 000 euros.



FIGURE 1.4 – Sensys Networks VDS240

1.4.2 Critique de l'existant

L'évaluation des trois solutions étudiées met en évidence des contraintes spécifiques liées à leur technologie, leur installation et leur coût.
Le Smartmicro UMRR-96, radar multicanal, exige un positionnement en hauteur et reste coûteux malgré une détection robuste .
Le FLIR TrafiCam x-stream™, caméra intelligente à vision embarquée, s'installe sur les feux de signalisation et nécessite une alimentation continue, bien que fiable, son prix élevé limite sa diffusion.
Le Sensys VDS240, basé sur des capteurs enfouis dans la chaussée, implique des travaux lourds lors de l'installation, ce qui augmente les coûts de déploiement.

Toutes ces solutions ont en commun une infrastructure lourde et un coût élevé.

Le tableau suivant illustre une comparaison entre les caractéristiques techniques et les contraintes de chaque solution.

Critère	**Smartmicro UMRR-96**	**FLIRTrafiCam x-stream™**	**Sensys VDS240**
Coût	Élevé 3 000 €	Très élevé 4 000 € - 13000 €[6]	Élevé 5 000 €
Fiabilité	Moyenne (IP67, -40°C à +85°C, 100g choc, 14g vibration)	Haute (IP67, -34°C à +80°C, conçu pour environnements urbains difficiles)	Haute(IP68,-40°C à +70°C, conçu pour résister aux chocs et vibrations
Confidentialité	Dépend de la gestion	Bonne	Complexe
Installation	Simple	Modérée	Simple
Maintenance	Facile	Facile	Difficile : implique de casser la chaussée.
Protocole de connexion	Ethernet / 4G	TCP/IP, NTCIP	RS-485 + 4G
Alimentation et Consommation	8W	15W	10W
Recharge et Autonomie	Branché en continu	Branché en continu	Branché en continu
Couverture	6 voies	4 voies	une seule voie

Table 1.1 – Tableau comparatif de l'existant

1.4.3 Solution proposée

Notre solution a pour objectif de couvrir une seule route avec une approche simple, fiable et économique pour la gestion du trafic et le calcul de la densité des véhicules en temps réel.
Elle s'appuie sur des capteurs de distance tels que le VL53L0X et des capteurs à ultrasons, placés sur le bord de la route pour détecter le passage des véhicules. Les données collectées seront ensuite envoyées à un serveur local via Wi-Fi et MQTT.
La solution sera également équipée d'un système d'alimentation et de recharge autonome, garantissant son fonctionnement continu.
L'architecture adoptée est centralisée, où les nœuds se limitent à la collecte des données, tandis que le traitement et le stockage sont assurés par un serveur central. Cette solution simplifie l'installation, réduit les coûts matériels et facilite la maintenance du système.

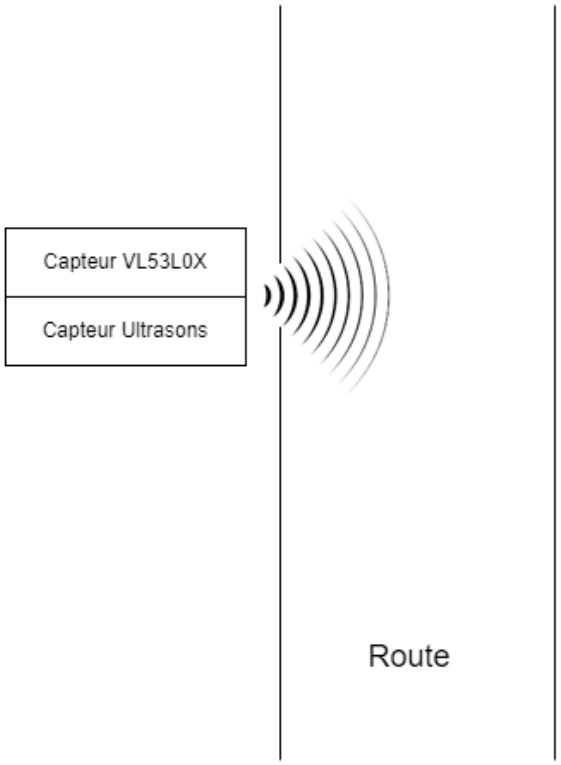


FIGURE 1.5 – Disposition des capteurs en bord de route

1.5 Méthodologie de travail et planification

1.5.1 Méthodologie de gestion de projet

La méthodologie de travail représente un ensemble de de méthodes et d'approches qui sont adoptées pour diviser les étapes et gérer le temps de travail sur le projet afin d'organiser le travail de meilleure manière. La méthodologie en spirale est considérée comme l'une des plus utilisées et des plus célèbres car elle est plus complète et a la capacité d'être adaptée au développement de projets.

Modèle en spirale

Le modèle en spirale est une approche éprouvée du développement logiciel qui combine les aspects itératifs des modèles incrémentaux et les éléments du modèle en cascade avec l'avantage de réduire les risques dans le développement logiciel, ce qui peut entraîner une augmentation des coûts globaux, ainsi que davantage d'efforts et de retards dans l'achèvement.
Par conséquent, un projet logiciel suit en permanence le cycle du modèle en spirale jusqu'à l'état final, en passant généralement par les quatre étapes suivantes :

1. Définition des objectifs et exigences

Cette étape consiste à définir les objectifs du projet, qui sont divisés en étapes individuelles du processus de développement, telles que l'amélioration des performances ou l'extension des fonctions. Il faut également déterminer le cadre général et le temps de travail requis.

2. Évaluation des risques et analyse
L'étape suivante consiste à identifier les zones d'incertitude, c'est-à-dire à anticiper les éventuels obstacles qui peuvent menacer l'avancement du projet de développement, puis à élaborer des plans de solutions ou des tests pour les éviter.

3. développement et contrôle de l'état intermédiaire
À cette étape, qui suit l'analyse des risques, un prototype ou une version du produit sera créé. C'est l'étape qui comporte les risques restants. S'il existe des problèmes liés aux performances ou des problèmes de contrôle d'interface interne, par exemple, qui affectent le processus de développement, il est possible de suivre dans un premier temps une stratégie de développement évolutive, où le projet est défini plus précisément et les prototypes sont affinés.

4. Dans la dernière étape, nous présentons les résultats obtenus en suivant les étapes précédentes. Des notes importantes sont collectées concernant la réalisation de l'objectif du cours et des solutions sont trouvées si l'étape de développement précédente ne s'est pas déroulée comme prévu. Ensuite, nous commençons à planifier le prochain objectif et ses prochaines étapes.

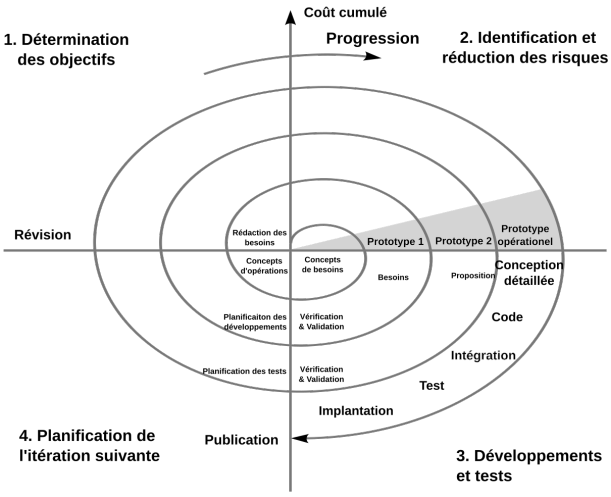


FIGURE 1.6 – Représentation du modèle en spirale

1.5.2 Méthodologie de conception

La méthodologie de conception UML (Unified Modeling Language) est une approche systématique de l'analyse, de la conception et de la modélisation d'un système logiciel qui utilise un ensemble de représentations et d'éléments conceptuels et les combine de diverses manières pour former des diagrammes qui représentent les aspects statiques ou structurels du système, ainsi que des diagrammes comportementaux qui décrivent les aspects dynamiques du système. UML facilite également la communication entre les développeurs et les analystes en fournissant un langage simple, commun et cohérent qui nous permet de valider et de suivre clairement la conception du système pour l'améliorer et l'examiner.



FIGURE 1.7 – Logo d'UML

1.6 Conclusion

Au cours de ce chapitre, nous avons placé le projet dans son cadre général et présenté la problématique fondamentale de ce sujet, puis présenté notre propre solution, étudié d'autres solutions disponibles et mentionné la méthodologie suivie pour mener à bien ce projet.

chapitre 2

Analyse des besoins

2.1 Introduction

Dans ce deuxième chapitre, nous discuterons des besoins fonctionnels et non fonctionnels Tionnelles de ce projet. Nous présenterons également les acteurs dans leur relation avec le système et pour comprendre comment ces deux derniers interagissent entre eux, nous mentionnerons les différents cas d'utilisation. Grâce à cette approche, nous aurons une idée claire du fonctionnement du projet de gestion du Traffic.

2.2 Analyse des besoins

2.2.1 Identification des acteurs

Les acteurs de notre solution sont :

- Administrateur : Il s'occupe de surveiller le système de gestion du trafic, d'extraire les diagrammes graphiques, de visualiser les données actuelles et historiques, ainsi que de recevoir des alertes et des notifications. Il est également responsable du calibrage des capteurs et de la configuration des routes et des nœuds du système.

- Technicien embarqué : Il supervise l'installation, la maintenance et le calibrage des capteurs sur le terrain. Il est chargé de configurer les paramètres des capteurs et de veiller à leur bon fonctionnement.

2.2.2 Besoins fonctionnels

Dans cette section, nous mentionnerons les besoins fonctionnelles du système, qui définissent et clarifient les tâches et la méthode de fonctionnement qui conduisent à son opérationnalisation.

1- Détecter le passage de vehicules en temps réel

2- Filtrer et valider les données localement

- Fusionner les données de deux nœuds .

- Filtrer les données.

3- Transférer les données vers le cloud en temps réel.

4- Afficher des statistiques en temps réel

- Fournir des graphiques montrant la répartition de la densité par heure, jour et mois.

5- Ajouter des routes et configurer les nœuds au sein du système.

6- Afficher des notification

- Diffuser des notifications sur le tableau de bord, fournissant un récapitulatif rapide des chiffres de densité de trafic à différents moments.

2.2.3 Besoins non fonctionnels

- Fiabilité : Le système doit garantir un fonctionnement optimal et continu, en assurant une détection précise et complète des véhicules, afin de maintenir la performance du système sur le long terme.

Facilité d'utilisation : Le système doit être simple à intégrer et à configurer, permettant une mise en place rapide et sans complexité, même pour les utilisateurs non-experts.

- Coût : Le système doit avoir un prix abordable, permettant une adoption et une intégration dans des infrastructures de petite taille ou dans des villes ayant un budget limité .

- Performance : Le système devrait permettre un fonctionnement rapide et cohérent a detecter tous les vehicules et transmettre les données.

- Scalabilité : Le système doit être facile d'ajouter de nouveaux capteurs ou nœuds.

2.3 Spécification des besoins

2.3.1 Cas d'utilisation global

Voici le diagramme global du système :

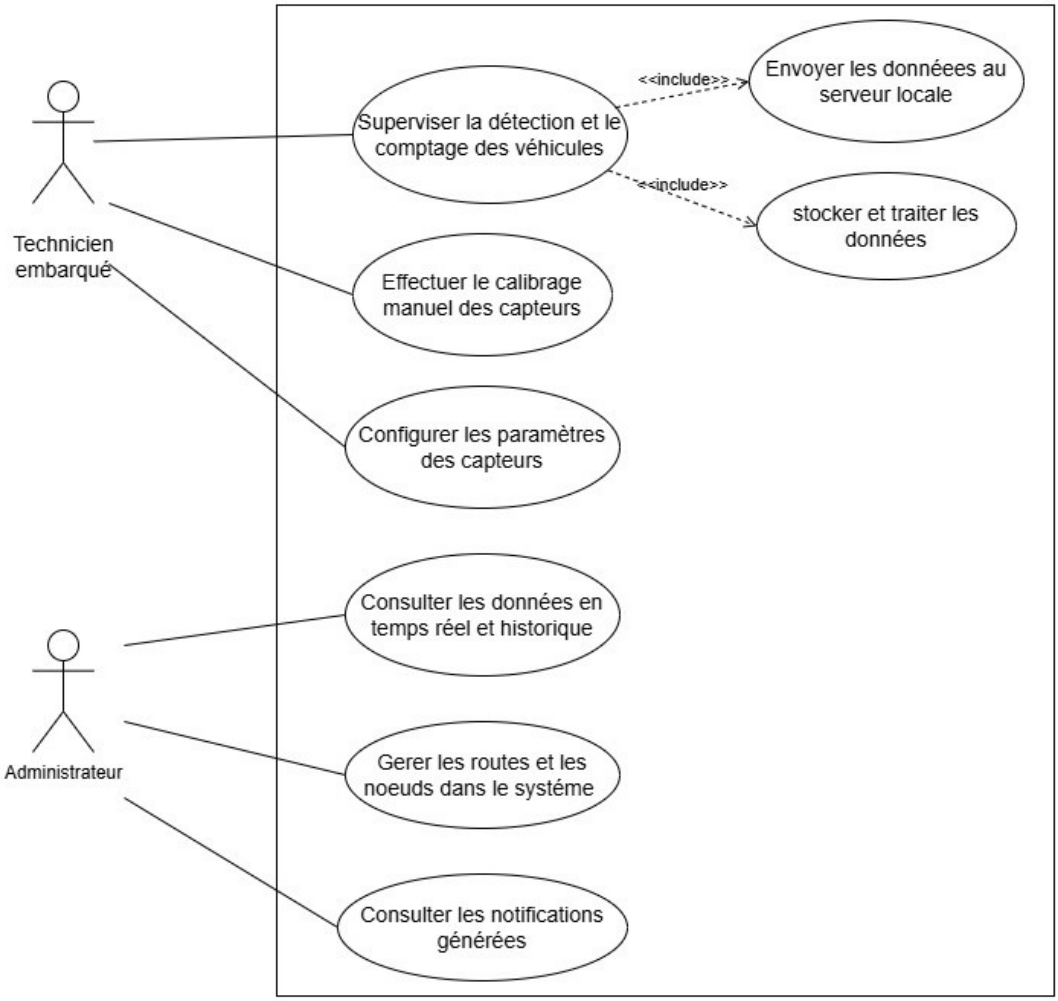


FIGURE 2.8 – Diagramme de cas d'utilisation global

2.3.2 les cas d'utilisation détaillées

2.3.2.1 Cas d'utilisation « Superviser la détection des véhicules »

La figure 2.9 illustre la description graphique de la cas d'utilisation «Superviser la detection des véhicules».

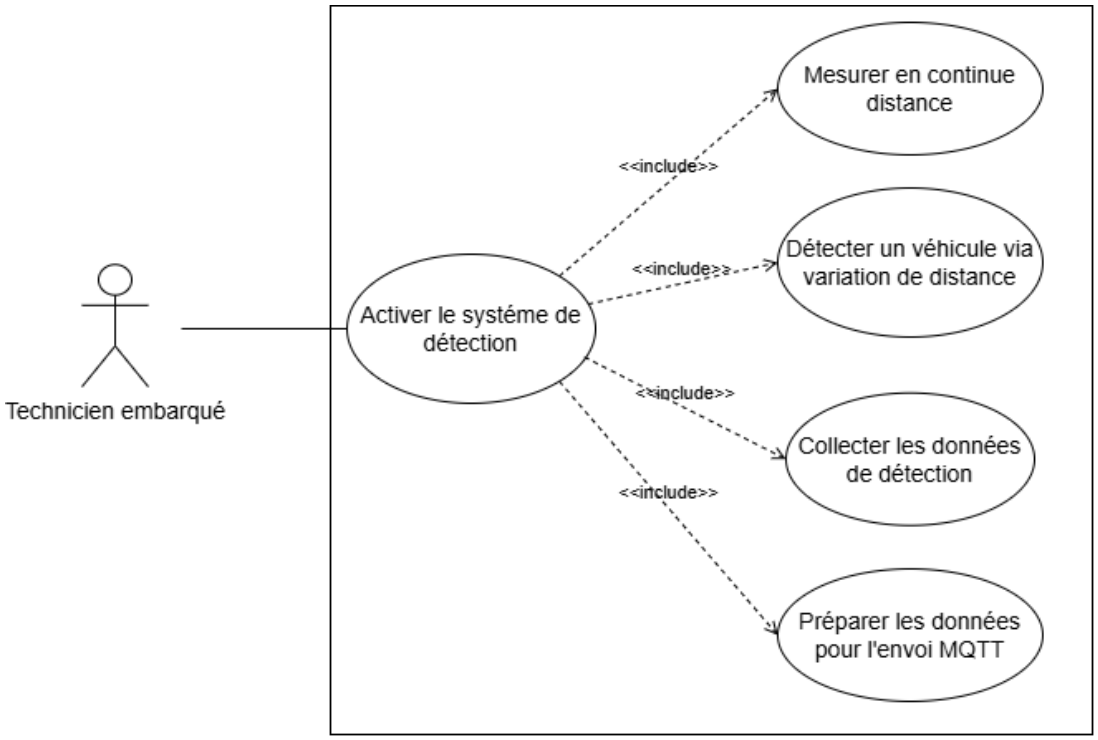


FIGURE 2.9 – Diagramme de cas d'utilisation « Superviser la détection des véhicules »

Le tableau 2.2 illustre la description textuelle de la d'utilisation ««« Superviser la detection des véhicules »».

Intitulé	Superviser la détection et le comptage
Acteur	Administrateur
Préconditions	L'Administrateur a configuré les capteurs de distance (installés, calibrés).
Scénario principal	— L'Administrateur active le système de détection via une interface ou un script. — Les capteurs mesurent en continu la distance pour détecter les véhicules (variation de distance). — Les capteurs enregistrent : comptage, horodatage, ID nœud, distance. — Les données sont préparées pour l'envoi MQTT.
Postconditions	Les données sont prêtes pour l'envoi MQTT.

TABLE 2.2 – Description textuelle du cas d'utilisation «Superviser la detection des véhicules»

2.3.2.2 Cas d'utilisation «Visualiser les données»

La figure 2.10 illustre une description graphique de cas d'utilisation «Visualiser les données»

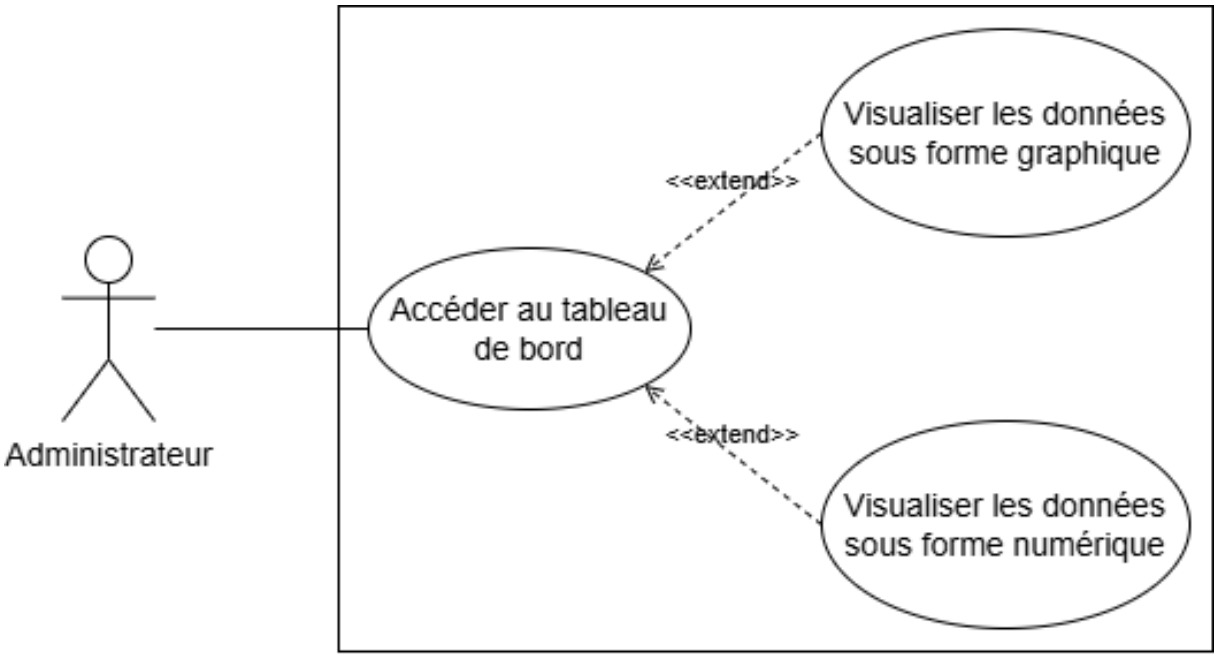


FIGURE 2.10 – Diagramme de cas d'utilisation «Visualiser les données»

Le tableau 2.3 illustre une description textuelle de cas d'utilisation «Visualiser les données»

Intitulé	Consulter les données via le tableau de Bord
Acteur	Administrateur
Préconditions	L'administrateur est connecté au système
Scénario principal	— L'Administrateur accède au tableau de bord . — Le tableau de bord affiche les données sous forme de graphiques/comptages. — Le tableau de bord se met à jour automatiquement.
Postconditions	Les données sont affichées avec succès

TABLE 2.3 – Description textuelle du cas d'utilisation «Visualiser les données»

2.3.2.3 Cas d'utilisation «Gérer les routes et les noeuds»

La figure 2.11 illustre une description graphique de cas d'utilisation «Gérer les routes et les noeuds»

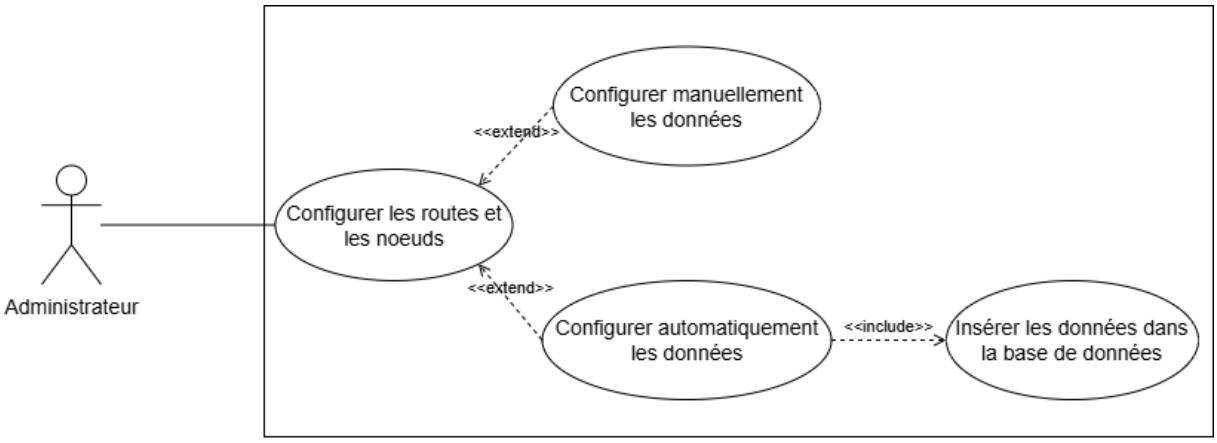


FIGURE 2.11 – Diagramme de cas d'utilisation «Gérer les routes et les noeuds»

Le tableau 2.4 illustre une description textuelle de cas d'utilisation «Gérer les routes et les noeuds»

Intitulé	Ajouter des routes et nœuds
Acteur	Administrateur
Préconditions	L'Administrateur a accès au script backend
Scénario principal	— L'Administrateur exécute le script backend. — L'Administrateur fournit une configuration contenant : Les noms des routes, Les localisations GPS des nœuds . — Le script lit cette configuration et insère les données dans la base de données.
Postconditions	Les routes et nœuds sont ajoutés avec succées au tableau de Bord

TABLE 2.4 – Description textuelle du cas d'utilisation «Gérer les routes et les noeuds»

2.4 Conclusion

Dans ce chapitre, nous avons approfondi les besoins fonctionnels et non fonctionnels de ce projet. Nous avons également défini le cas d'utilisation global, puis répertorié les sous-cas d'utilisation afin de parvenir à une meilleure compréhension du fonctionnement du système.
Dans le chapitre suivant, nous discuterons en détail de la conception du projet.

chapitre 3

Conception

3.1 Introduction

Dans ce chapitre, nous modélisons la conception du système de surveillance de trafic via des diagrammes de séquence, illustrant les interactions pour la détection, le traitement des données et l'affichage en temps réel. Nous présentons aussi un schéma simplifié de la base de données pour gérer les routes et détections, clarifiant les flux et relations entre composants logiciels et matériels.

3.2 Structure globale du système

La structure globale de notre projet repose sur plusieurs aspects fondamentaux qui, une fois combinés, ont permis de concevoir une solution réalisable. Le projet intègre un volet de systèmes embarqués, chargé d'exécuter des tâches nécessitant une interaction directe avec les dispositifs, ainsi que de gérer l'alimentation et la recharge du système.

Il comprend également une couche backend, qui assure le traitement de la logique métier, la gestion des communications réseau, ainsi que l'interfaçage avec la base de données. Enfin, le front-end représente la couche de présentation, fournissant à l'utilisateur une interface graphique intuitive pour interagir avec le système.

La section suivante détaillera les fonctionnalités et le rôle de chaque composant de manière approfondie.

3.2.1 Composants de l'architecture

La figure 3.12 illustre l'architecture matérielle du système de détection de trafic, mettant en évidence l'intégration des capteurs avec le microcontrôleur ESP32. L'alimentation, fournie par un système continu connecté à une source externe, alimente deux capteurs VL53L0X (via I2C) et un capteur à ultrasons (via GPIO), tous reliés à l'ESP32. Ce dernier transmet les données collectées au serveur local via une connexion Wi-Fi, assurant une communication efficace pour le traitement et l'analyse des données en temps réel.

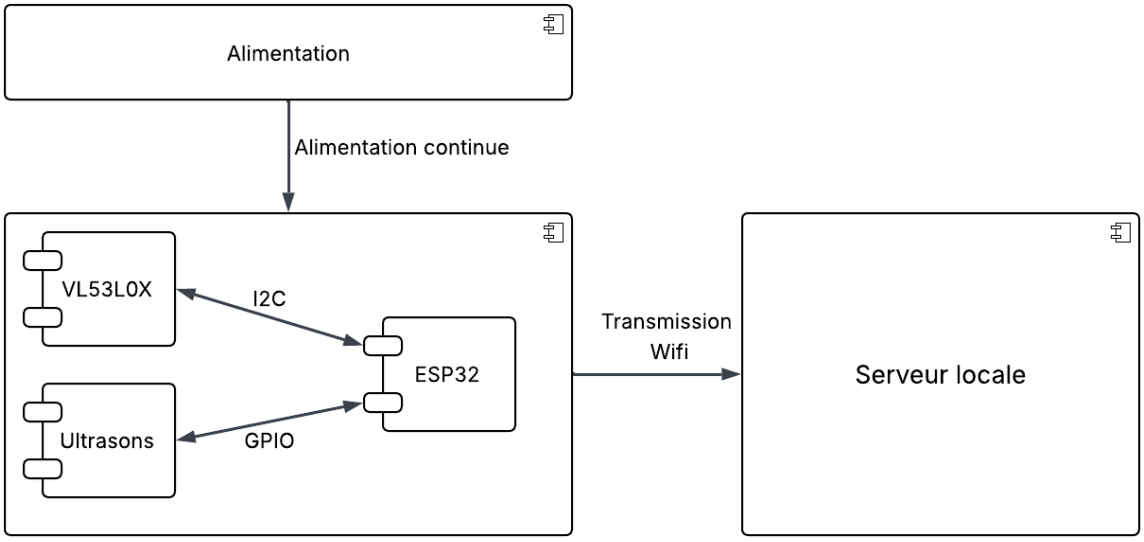


FIGURE 3.12 – Diagramme de composants du système

- **Systéme embarqué**

Le système embarqué dans ce projet représente la base, car c'est lui qui détecte le mouvement des véhicules sur la route.
Dans le cas où un véhicule passant est détecté , il collecte les données et les envoie en temps réel via Wi-Fi au serveur central.

Dans cette partie, le système est divisé en plusieurs sous-ensembles essentiels. Tout d'abord, on retrouve les capteurs, chacun étant soigneusement calibré pour garantir une collecte de mesures fiables. Ensuite, ces données sont transmises à un microcontrôleur, qui assure un traitement local de l'information, où il filtre, analyse et formate les données recueillies en vue de leur transmission. Par ailleurs, un circuit d'alimentation et de recharge est intégré au système ; il a pour rôle de fournir une tension stable aux différents composants et d'assurer une recharge efficace de la batterie, permettant ainsi au dispositif de fonctionner de manière autonome, continue et dans de bonnes conditions.

- **Backend**

Le backend est la partie où les informations sont extraites en vue d'un traitement ultérieur, incluant les calculs nécessaires. Il assure également le stockage des données de manière organisée, ce qui permet d'y accéder facilement et de les exploiter à des fins d'analyse et de statistiques. Grâce à lui, il est possible de communiquer avec le frontend et de lui communiquer des informations en temps réel.

- **Frontend**

Le front-end est la partie visible par l'utilisateur et avec laquelle il interagit principalement. Dans ce projet, il affiche des informations sur la densité de trafic actuelle à des points spécifiques déterminés en fonction des emplacements des nœuds. Il fournit également des diagrammes plus détaillés qui garantissent une expérience visuelle fluide et

intuitive.

3.2.2 Diagramme de déploiement

Un diagramme de déploiement est utilisé pour créer un modèle illustratif de la structure physique d'un système. Il fournit une image claire des relations entre les composants logiciels et matériels du système, d'une part, et la distribution physique du traitement, d'autre part.
Comme le montre la figure 3.13 ci-dessous

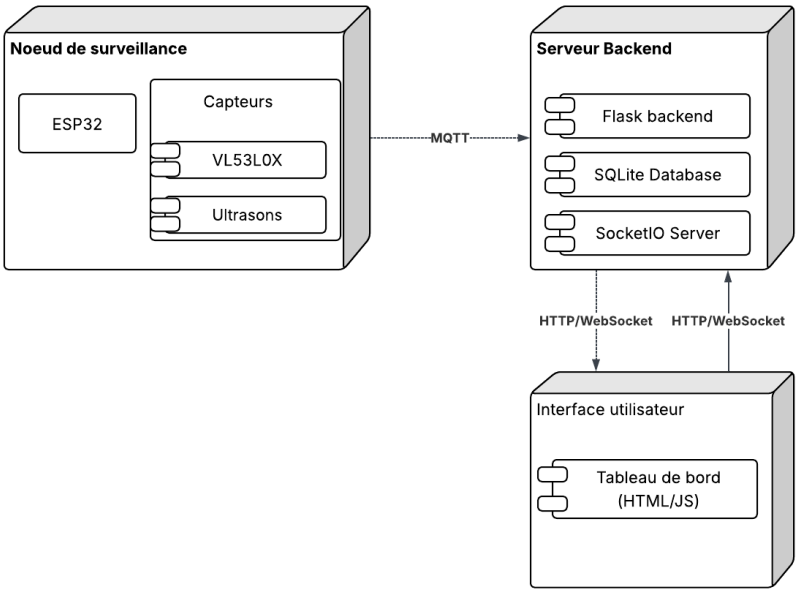


FIGURE 3.13 – Diagramme de déploiement global

3.2.3 Protocoles et interfaces de communication

Le protocole MQTT (Message Queuing Telemetry Transport) est utilisé dans cette solution afin d'assurer une communication fiable entre le microcontrôleur et le serveur. Conçu spécifiquement pour l'Internet des objets (IoT) et les systèmes embarqués, ce protocole se distingue par sa légèreté et son efficacité. Il repose sur un modèle de communication de type publication/abonnement, où l'ESP32 agit en tant qu'éditeur, publiant les données vers un MQTT Broker avec un niveau défini de Qualité de Service (QoS). Le serveur Flask, quant à lui, joue le rôle d'abonné et reçoit les données publiées.

Parmi les caractéristiques techniques majeures du protocole figure la Qualité de Service, qui permet de définir le degré de fiabilité de la transmission des messages. MQTT est également apprécié pour sa faible empreinte, notamment grâce à une taille minimale d'en-tête limitée à seulement 2 octets[7].

La figure 3.14 illustre une architecture MQTT (Message Queuing Telemetry Transport), où un courtier central facilite une communication asynchrone entre plusieurs clients.

Les clients peuvent publier des messages vers le courtier et s'abonner à des sujets spécifiques, permettant ainsi un échange de données efficace au sein du système.

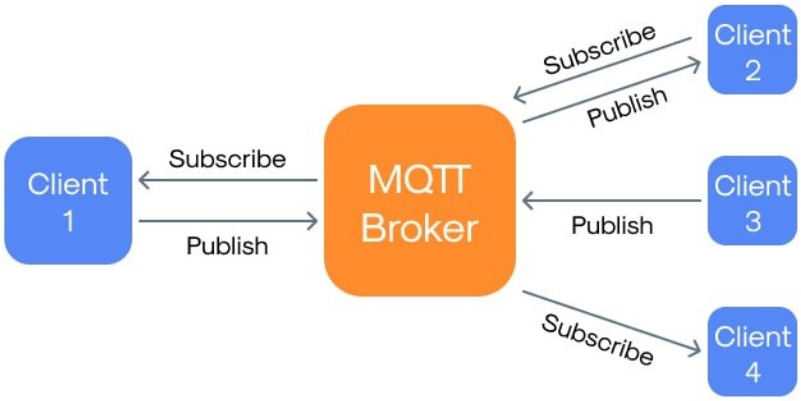


FIGURE 3.14 – Diagramme de fonctionnement du protocole MQTT

3.3 Modélisation dynamique du système

Un diagramme d'activité se concentre spécifiquement sur la modélisation du flux de traitement et de contrôle de l'information d'une activité à une autre, c'est-à-dire qu'il fournit une explication du comportement d'un système en montrant la séquence d'actions dans un processus en son sein. Il se concentre sur les éléments d'un système afin de visualiser, de spécifier ou de documenter le comportement de cet élément.

Dans l'étape suivante, nous discuterons du diagramme d'activité général et identifierons les sous-diagrammes pour acquérir une compréhension plus complète de la manière dont l'information circule au sein des éléments du système.

3.3.1 Diagramme d'activité global

La figure 3.15 représente le diagramme d'activité global du système, illustrant le fonctionnement complet depuis la détection d'un véhicule jusqu'à l'affichage sur le dashboard. La partie embarquée détecte le passage d'un véhicule et envoie les données via Wi-Fi et MQTT vers un broker. Un backend, abonné au broker, traite et stocke ces données dans une base de données. Enfin, un frontend interroge cette base pour afficher les résultats en temps réel.
Les figures suivantes détaillent les étapes de détection, la connectivité réseau, et le traitement des données côté serveur.

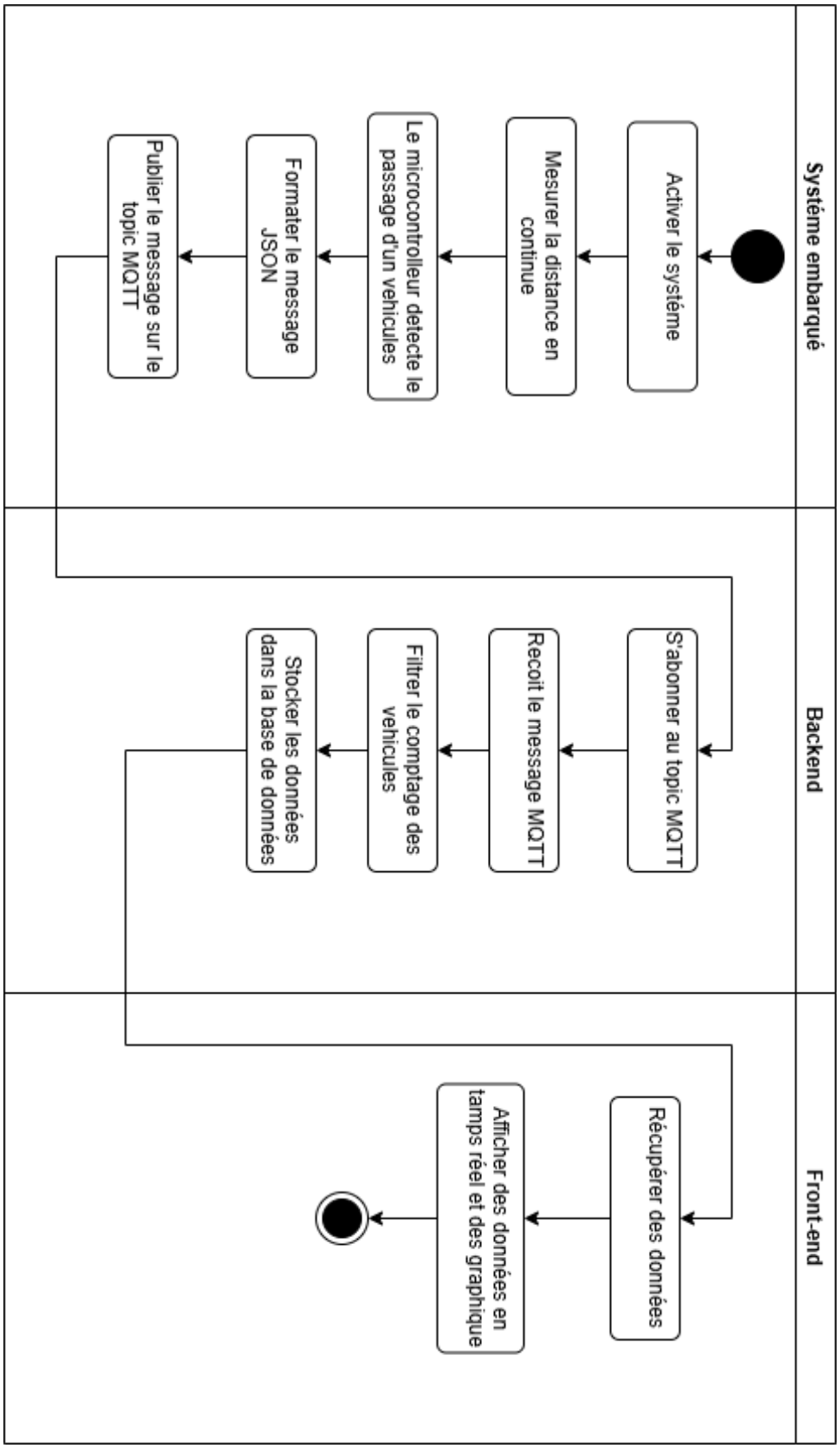


FIGURE 3.15 – Diagramme d'activité global

3.3.2 Diagramme d'activité « Connectivité WiFi et MQTT »

Ce diagramme décrit le processus par lequel le microcontrôleur établit la communication réseau nécessaire à l'envoi des données détectées.
Tout d'abord, le système initialise le module WiFi en utilisant les identifiants du réseau configurés. Ensuite, il tente de se connecter à ce réseau. Si cette connexion échoue, une nouvelle tentative est automatiquement lancée jusqu'à réussite.
Une fois connecté au WiFi, le système initialise le client MQTT en spécifiant l'adresse du broker. Si la connexion MQTT échoue, le système recommence aussi cette étape. Quand les deux connexions sont établies, le système devient prêt à publier les données.

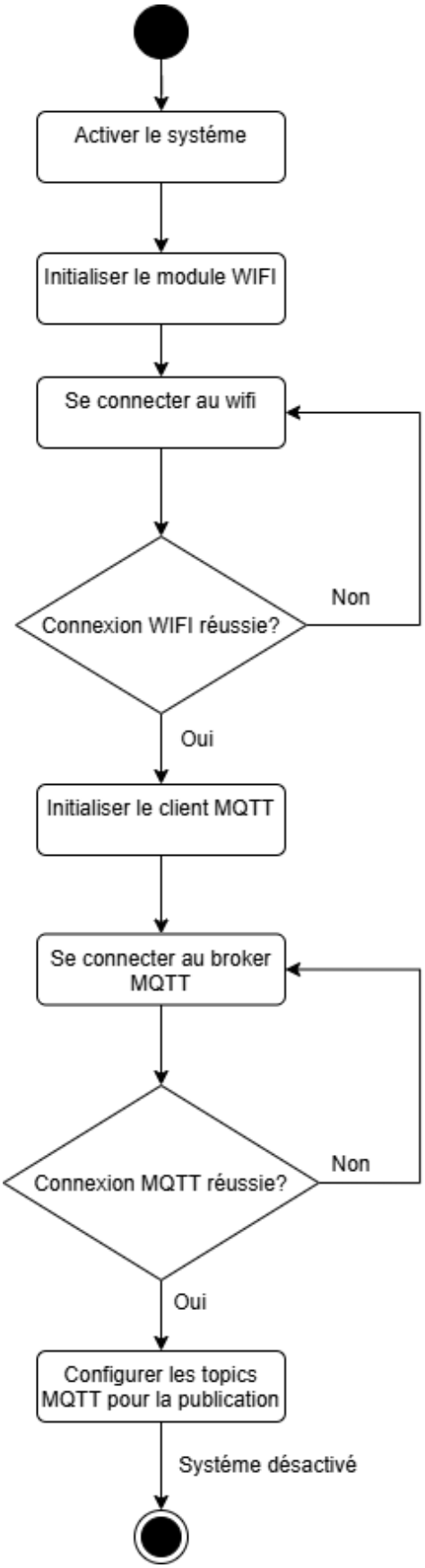


FIGURE 3.16 – Diagramme d'activité « Connectivité WiFi et MQTT »

3.3.3 Diagramme d'activité « Detection de vehicule »

la figure 3.17 décrit le fonctionnement interne du programme embarqué exécuté sur le microcontrolleur.
Le diagramme détaille la logique de détection du passage d'un véhicule à l'aide des capteurs de distance, en distinguant les différents états (WAITING, DETECTING, COUNTED) par lesquels passe le système.
Chaque transition dépend des valeurs mesurées et du temps écoulé, assurant une détection fiable avant l'envoi des données via MQTT. Cette activité représente le cœur de la logique locale avant la communication avec les autres composants du système.

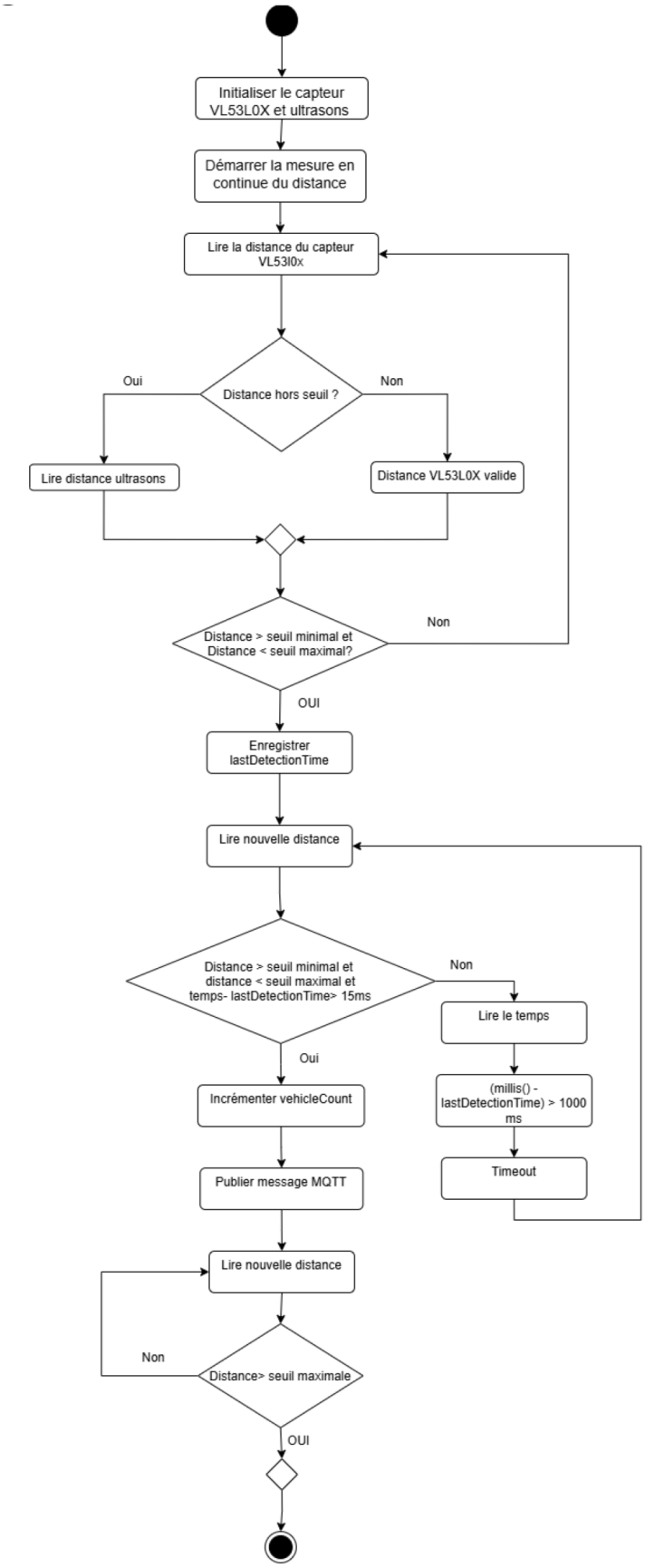


FIGURE 3.17 – Diagramme d'activité « Detection des vehicules »

3.4 Diagrammes de séquence

Un diagramme de séquence permet de modéliser, de manière chronologique, les interactions entre différents composants d'un système, en mettant en évidence l'enchaînement précis des messages échangés.
Il illustre non seulement le flux d'informations entre les entités, mais aussi les structures de contrôle mises en œuvre, telles que les boucles ou les alternatives, pour refléter fidèlement le comportement dynamique du système étudié.

3.4.1 Diagramme de séquence « Détection de véhicule »

Le diagramme de séquence de la figure 3.18 illustre le processus de détection d'un véhicule basé sur les capteurs VL53L0X et ultrasoniques connectés à un ESP32. Après l'initialisation des capteurs, l'ESP32 effectue une lecture continue de la distance mesurée. Une calibration est appliquée afin de corriger les éventuelles erreurs de mesure. L'ESP32 surveille ensuite la stabilité de la distance en passant entre les états. Enfin, il publie un message JSON contenant les informations de détection via MQTT.

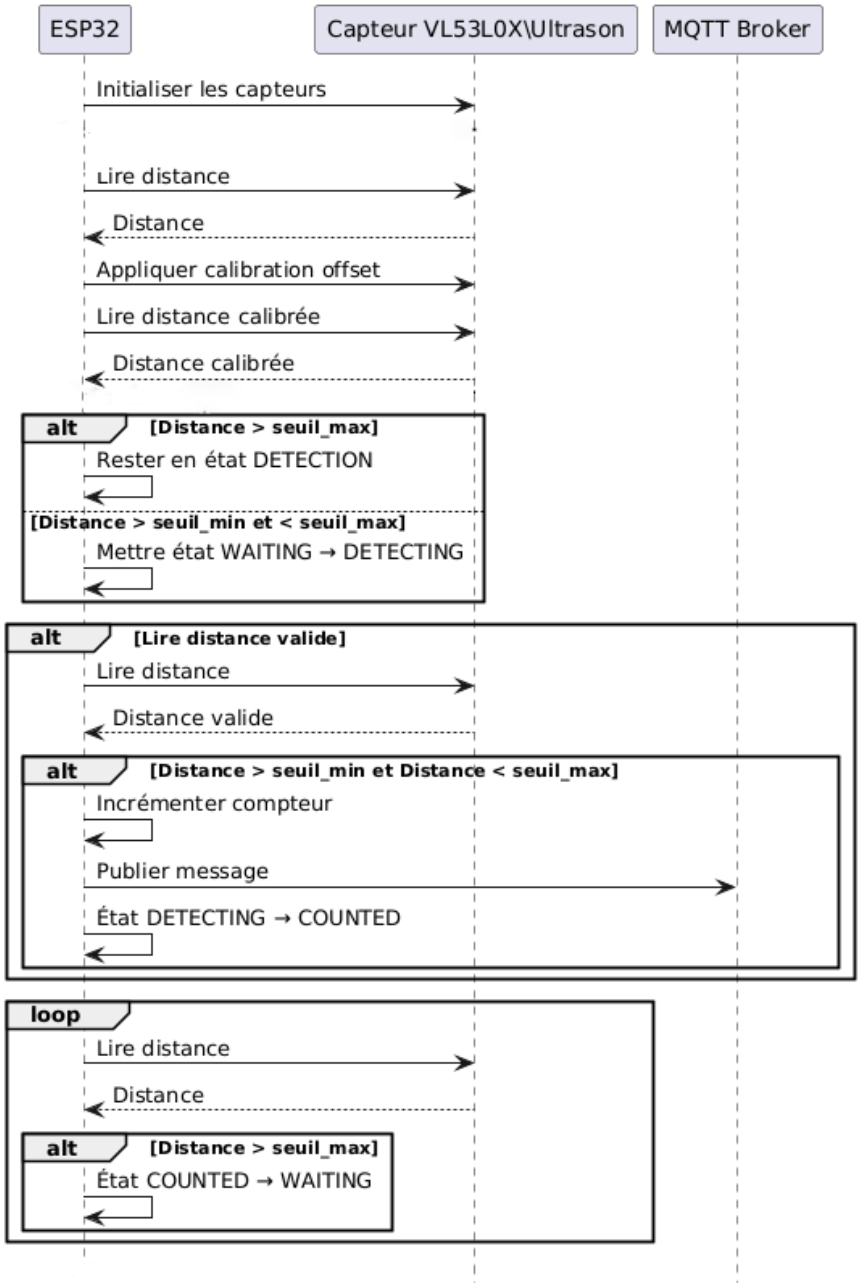


FIGURE 3.18 – Diagramme de séquence «Détection de véhicule »

Diagramme de séquence « Detection de vehicule » en mode interruption

La détection des véhicules peut être optimisée en utilisant le capteur VL53L0X en mode interruption, permettant au système de réagir uniquement lorsqu'un véhicule est détecté.
Ce mode fonctionne en surveillant en temps réel la distance mesurée, et dès qu'un objet, comme un véhicule, franchit un seuil prédéfini, une interruption est déclenchée, signalant immédiatement sa présence sans avoir besoin de surveillance continue. Le diagramme de séquence présenté illustre clairement ce fonctionnement, en détaillant les échanges entre le capteur et le microcontrôleur lors de la détection.

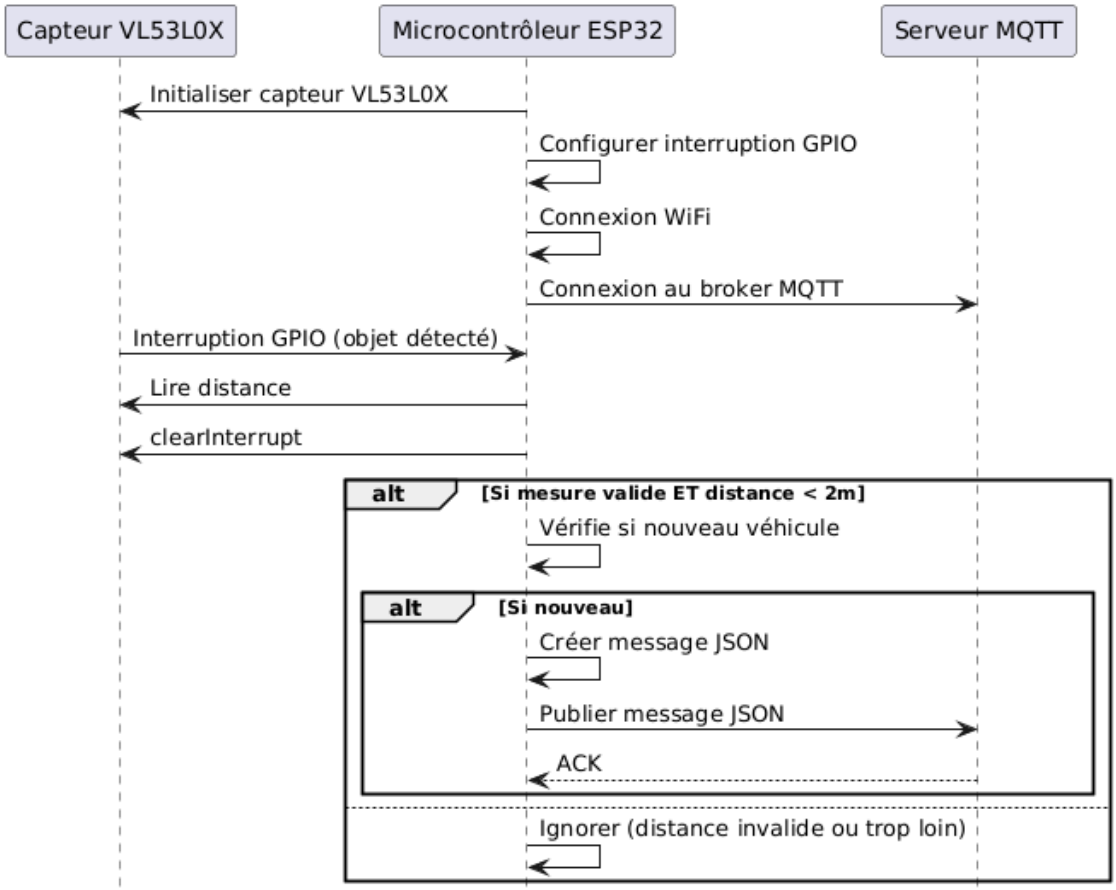


FIGURE 3.19 – Diagramme de séquence « Detection de vehicule » en mode interruption

3.4.2 Diagramme de séquence « Réception et traitement des données »

Le diagramme de séquence ci-dessous illustre le traitement des messages côté serveur Flask après la réception d'un message JSON depuis le broker MQTT. Le backend Flask, abonné au topic, vérifie d'abord si la détection a déjà été prise en compte afin d'éviter les doublons. Si un nouveau véhicule est détecté, le compteur global est incrémenté et les données sont enregistrées dans la base de données. Le diagramme suivant détaillera la conditions exacte utilisée pour filtrer les doublons.

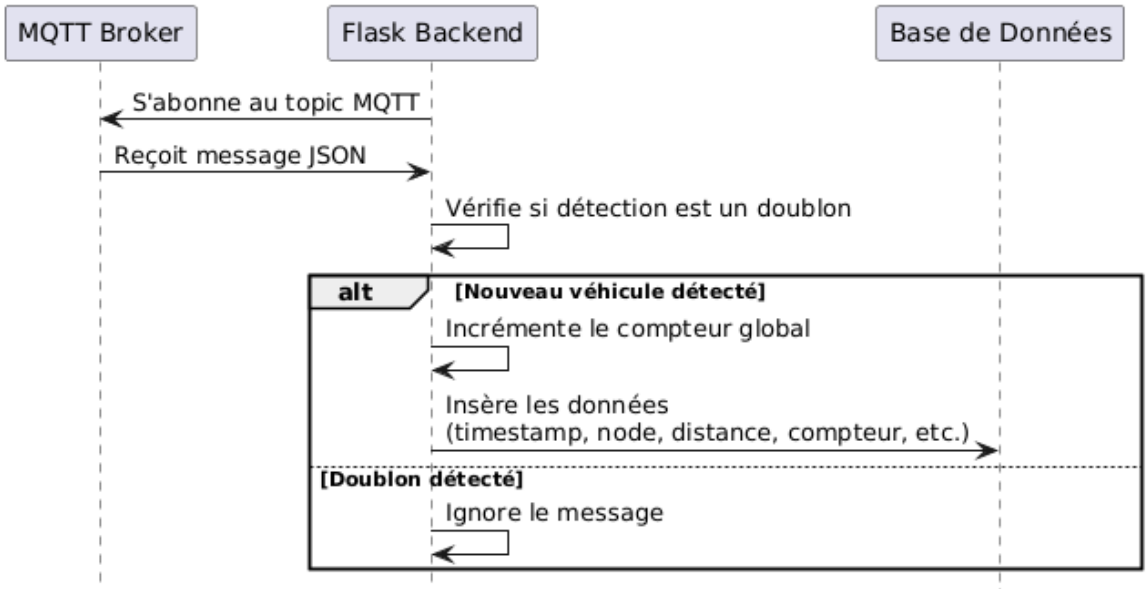


FIGURE 3.20 – Diagramme de séquence « Réception et traitement des données »

3.4.3 Diagramme de séquence « Filtrage des Doublons »

Ce diagramme illustre la logique de synchronisation entre deux capteurs (Node1 et Node2) afin d'éviter un double comptage d'un même véhicule. Les deux capteurs envoient leurs messages JSON au backend via le broker MQTT. Le backend compare alors les timestamps et les distances reçus. Si les données sont suffisamment proches dans le temps et l'espace, une seule détection est comptabilisée et enregistrée dans la base de données.

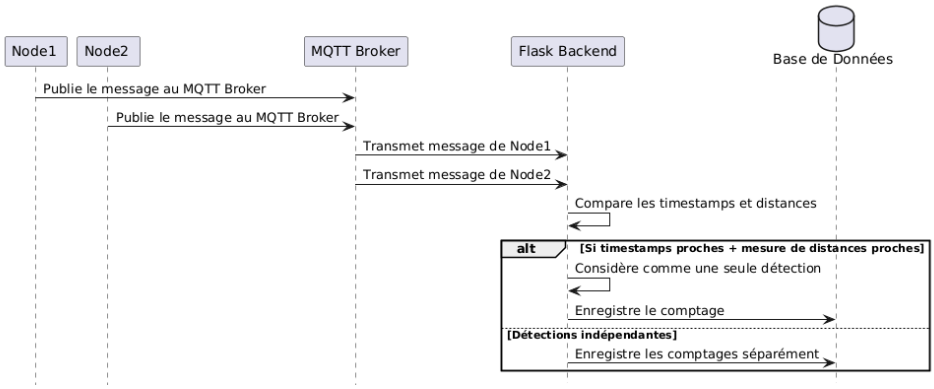


FIGURE 3.21 – Diagramme de séquence « Filtrage des Doublons »

3.4.4 Diagramme de séquence « Affichage sur le tableau de bord »

Ce diagramme illustre le mécanisme de réception des messages émis par le backend au moyen d'une connexion Socket.IO, garantissant une transmission des données en temps réel.
Ce dispositif permet une mise à jour dynamique et immédiate des représentations visuelles, telles que les figures, les tableaux, les graphiques, ainsi que les notifications.

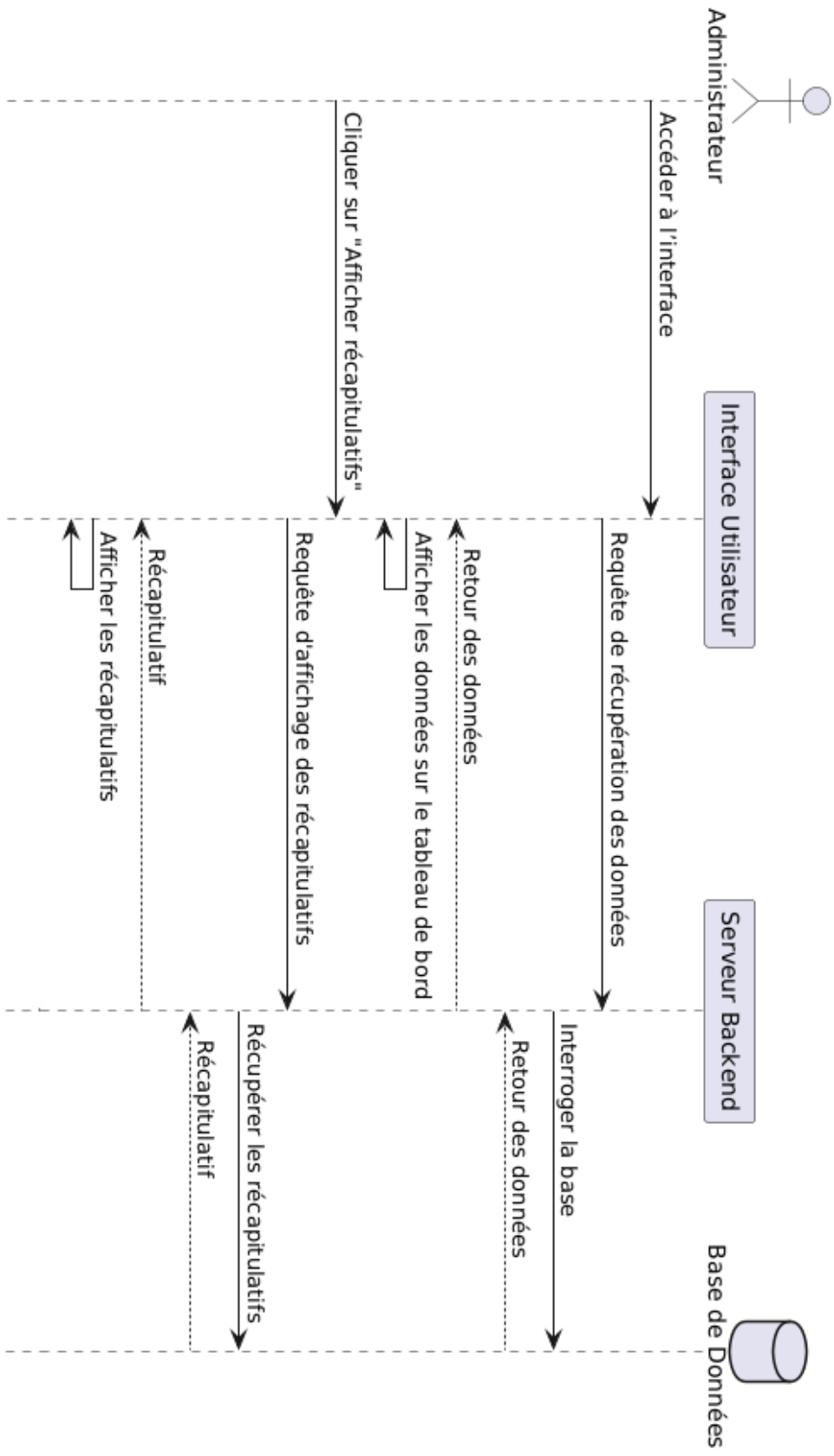


FIGURE 3.22 – Diagramme de séquence « Affichage sur le tableau de bord »

3.4.5 Diagramme de séquence « Configuration des routes »

Ce diagramme démontre l'interaction entre l'administrateur, le front-end, le back-end et la base de données pour intégrer et visualiser efficacement les nouveaux nœuds de capteur.
Ce processus permet d'insérer les données correspondantes dans la base de données, tout en mettant à jour le backend pour assurer la cohérence du système. Une fois l'insertion effectuée, le tableau de bord est automatiquement synchronisé afin d'afficher les nouvelles informations, offrant ainsi une vue en temps réel du réseau.

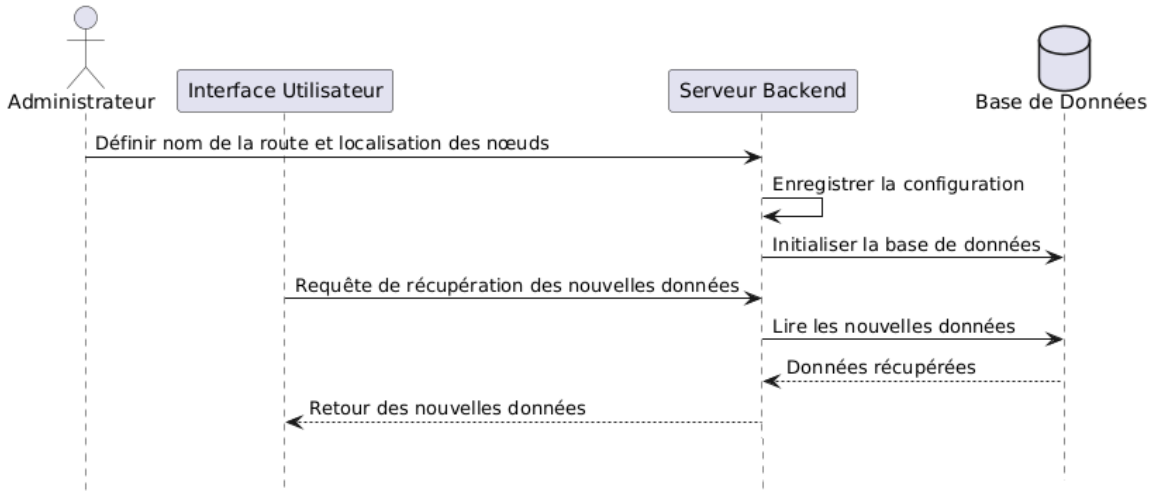


FIGURE 3.23 – Diagramme de séquence « Configuration des nœuds et des routes »

3.4.6 Étude d'une solution alternative basée sur l'interruption

Dans ce projet, le système de détection de distance peut être implémenté selon deux modes :
le mode polling, consistant à interroger périodiquement le capteur dans la boucle principale pour lire les valeurs de distance, ou le mode interruption matérielle, reposant sur un signal déclenché par le capteur lorsqu'un changement de distance est détecté.
Le mode interruption permet une meilleure réactivité et économise du temps processeur par rapport au polling.
Pour permettre cette détection via interruption, une résistance de pull-up [8] a été ajoutée manuellement sur l'entrée GPIO, conformément aux recommandations de la datasheet, qui indique que la sortie du capteur est en niveau bas par défaut (état actif bas). L'objectif était de forcer le niveau haut en condition normale et de détecter une transition vers le bas.
Cependant, cette configuration n'a pas permis le déclenchement attendu de l'interruption.

Dans la suite, nous présentons des diagrammes décrivant le flux de traitement dans le cas où l'approche par interruption serait pleinement fonctionnelle, afin de mettre en valeur la logique de cette alternative.

Diagramme d'activité global

La figure 3.24 représente le diagramme d'activité globale en mode interruption .
Il explique le déroulement général du système depuis l'activation jusqu'à la désactivation en incluant la détection, le traitement, la transmission et l'affichage des données.

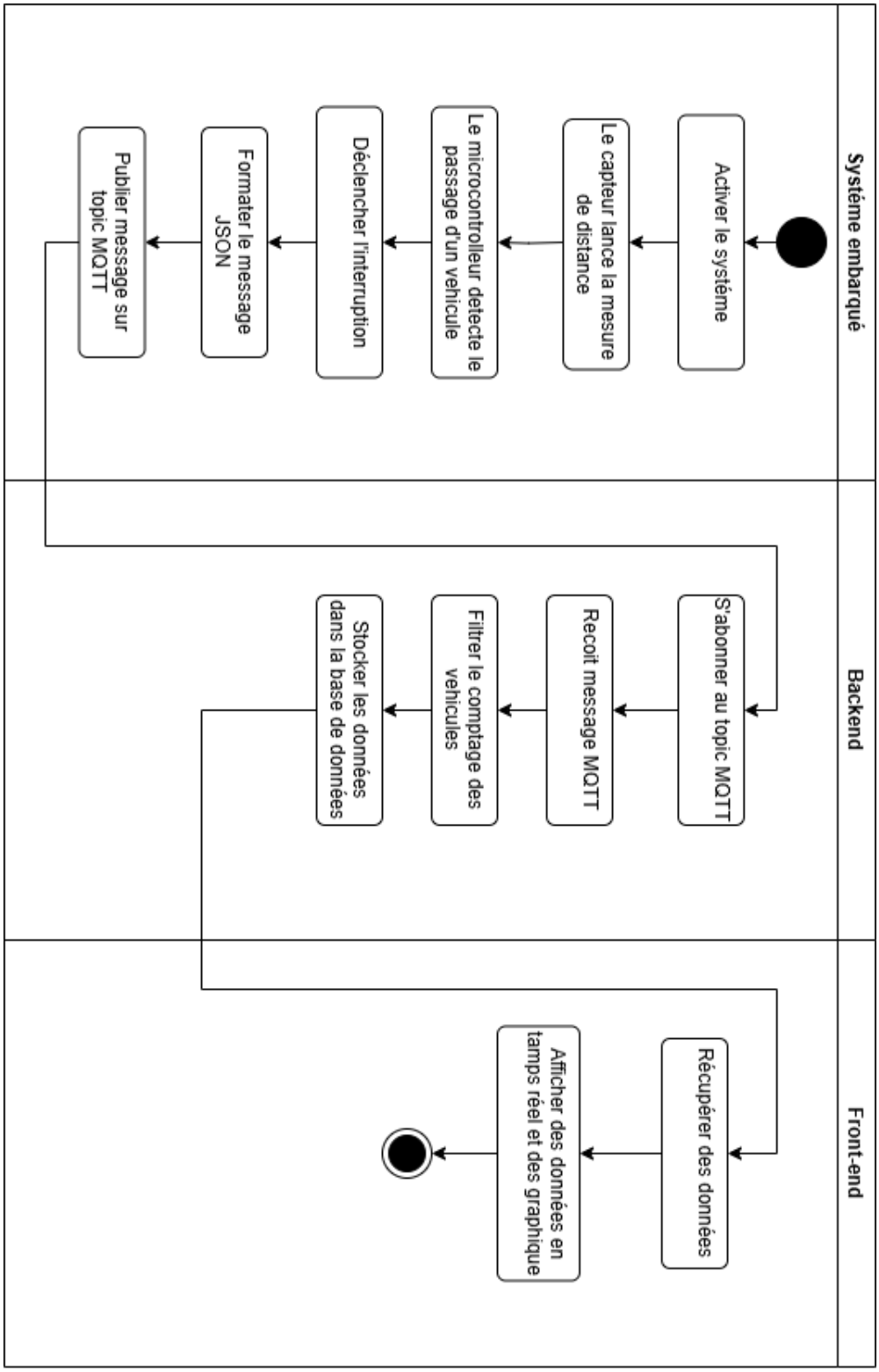


FIGURE 3.24 – Diagramme d'activité globale en mode interruption

3.5 Base de Données

Dans cette section, nous présentons le schéma simplifié de la base de données du système. Il centralise les données des véhicules sur des routes définies à travers deux tables principales : Routes, qui stocke les informations des routes, et History, qui enregistre les détections de véhicules avec leurs nœuds associés, distances et horodatages. La table History est liée à Routes par route-id, où une route peut avoir plusieurs détections, assurant la cohérence et permettant de calculer dynamiquement les historiques (journaliers, horaires, etc.) selon les besoins.

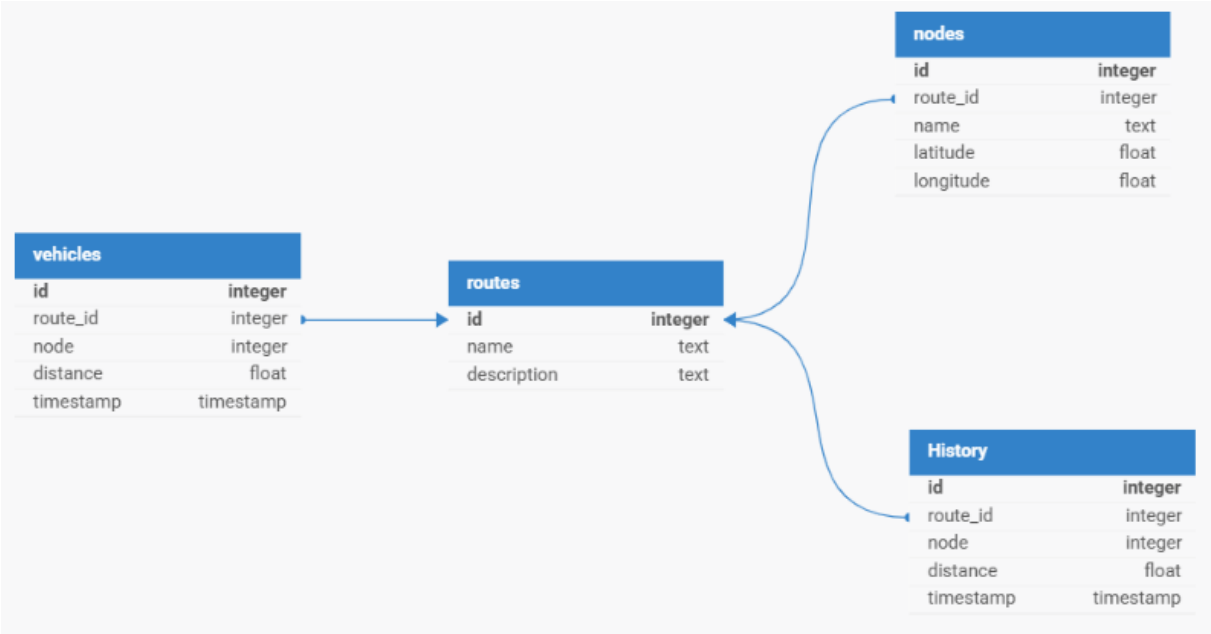


FIGURE 3.25 – Schéma Entity-Relationship (ER) de la base de données

3.6 Conclusion

Ce chapitre a détaillé la conception du système de surveillance du trafic à travers des diagrammes de séquence, illustrant les interactions entre capteurs, microcontrôleur, serveur Flask et interface utilisateur. Le schéma simplifié de la base de données, structuré autour des tables Routes et History, facilite la gestion et l'analyse des données. Cette modélisation met en évidence la logique opérationnelle du système, les optimisations possibles (comme le mode interruption) et prépare efficacement la phase de réalisation.

chapitre 4

Réalisation

4.1 Introduction

Dans ce chapitre, nous abordons la concrétisation du projet. Il commence par une présentation de l'environnement logiciel utilisé, en mettant en avant les bibliothèques et outils ayant permis le développement du système. L'environnement matériel est également présenté afin de situer le contexte global de mise en œuvre. Ensuite, les principales étapes de la réalisation sont détaillées, suivies d'une description des tests effectués pour évaluer le bon fonctionnement et la fiabilité de l'ensemble.

4.2 Environnement logiciel

4.2.1 Partie embarqué

- Arduino IDE

L'Arduino IDE est un environnement de développement open source destiné à la programmation de cartes à microcontrôleur telles que les Arduino (Uno, Mega, Nano) et les cartes compatibles comme les ESP32 et ESP8266.

Basé sur un langage dérivé du C/C++, il propose une interface simple pour écrire, compiler et téléverser du code via USB. L'IDE intègre également de nombreuses bibliothèques logicielles, facilitant l'utilisation de capteurs, d'actionneurs et de modules de communication pour le développement de systèmes embarqués.



FIGURE 4.26 – Logo arduino ide

4.2.2 Partie front-end

- Visual Studio

Visual Studio est un environnement de développement intégré (IDE) développé par Microsoft, utilisé pour créer des applications en C++, C, Python, et d'autres langages. Il offre des outils complets pour la programmation, le débogage et le déploiement, avec des fonctionnalités avancées comme IntelliSense, un débogueur intégré, et une large prise en charge des extensions via le Visual Studio Marketplace.

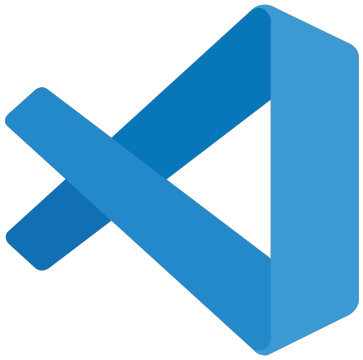


FIGURE 4.27 – Logo VS

- Html

HTML (HyperText Markup Language) est un langage de balisage standard utilisé pour structurer le contenu des pages web. Il permet de définir les éléments tels que les titres, paragraphes, images, liens, et autres composants d'une page. HTML sert de squelette à toute page web, organisant le contenu pour qu'il soit interprété et affiché correctement par les navigateurs.



FIGURE 4.28 – Logo html

◦ Java script

JavaScript est un langage de programmation interprété, principalement utilisé côté client dans les navigateurs web. Il permet de rendre les pages web interactives en manipulant le contenu HTML et le style CSS dynamiquement, en gérant les événements utilisateur, et en communiquant avec les serveurs pour charger ou envoyer des données sans recharger la page.



FIGURE 4.29 – Logo JS

◦ CSS

CSS est un langage de feuilles de style qui permet de contrôler la présentation visuelle des documents HTML. Il définit des règles pour la mise en forme des éléments (couleurs, polices, marges, dispositions, animations, etc.), séparant ainsi le contenu (HTML) de son apparence. CSS facilite la création de designs cohérents et adaptatifs sur différents supports.



FIGURE 4.30 – Logo CSS

4.2.3 Partie Backend

◦ Flask[17]

Flask est un micro-framework web open source écrit en Python. Il est conçu pour être léger et simple, offrant les fonctionnalités essentielles pour créer des applications web ou des API. Grâce à sa modularité, Flask permet d'ajouter facilement des extensions pour

gérer les bases de données, l'authentification ou d'autres besoins spécifiques. Il utilise le moteur de templates Jinja2 pour générer des pages dynamiques et dispose d'un serveur de développement intégré, facilitant le prototypage rapide.



FIGURE 4.31 – Logo Flask

◦ Python

Python est un langage de programmation interprété, polyvalent et réputé pour sa syntaxe claire et facile à apprendre. Il est utilisé dans de nombreux domaines, allant du développement web à l'analyse de données et à l'intelligence artificielle. Python supporte plusieurs paradigmes de programmation et possède une vaste bibliothèque standard, complétée par un écosystème riche de modules tiers, ce qui permet un développement rapide et efficace sur toutes les plateformes majeures.



FIGURE 4.32 – Logo python

◦ WebSocket[18]

WebSocket est un protocole de communication réseau qui permet d'établir une connexion bidirectionnelle et persistante entre un client et un serveur. Contrairement au protocole HTTP classique basé sur des requêtes ponctuelles, WebSocket maintient la connexion ouverte pour un échange de données en temps réel et à faible latence. Ce protocole est particulièrement adapté aux applications interactives comme les jeux en ligne, les chats ou les systèmes de notifications instantanées.



FIGURE 4.33 – Logo WebSocket

4.2.4 Bibliothèques utilisées :

VL53L0X.h

VL53L0X.h est une bibliothèque utilisée pour interfacer le capteur de distance laser VL53L0X avec des microcontrôleurs comme Arduino ou ESP32. Elle permet de mesurer des distances précises en utilisant la technologie ToF (Time of Flight) et simplifie la lecture des données du capteur pour des applications de détection et de mesure.

Socket.IO

Socket.IO est une bibliothèque qui facilite la communication en temps réel entre un client et un serveur. Elle repose sur le protocole WebSocket et gère automatiquement les reconnexions et la compatibilité entre différents navigateurs, ce qui la rend idéale pour les applications interactives nécessitant un échange instantané d'informations.

Paho.mqtt

Paho.mqtt est une bibliothèque cliente MQTT développée par Eclipse, utilisée pour implémenter des communications légères et efficaces dans des systèmes IoT. Elle permet aux appareils de publier et de s'abonner à des messages sur un broker MQTT, favorisant ainsi un échange asynchrone et fiable de données entre objets connectés.

4.2.5 Outils de modélisation des diagrammes et de rédaction

- **Overleaf**

Overleaf est un éditeur en ligne conçu pour la rédaction de documents scientifiques en LaTeX. Il permet de travailler à plusieurs en temps réel, de structurer du contenu technique avec précision (formules, figures, bibliographies) et de générer rapidement des fichiers PDF bien formatés, ce qui en fait un outil précieux pour la documentation académique et technique.



FIGURE 4.34 – Logo Overleaf

- **Draw.io**

Draw.io est une application web qui permet de créer facilement des diagrammes, des organigrammes, des schémas de systèmes ou encore des plans d'architecture logicielle. Son interface intuitive et ses nombreuses formes intégrées en font un outil adapté à la modélisation de projets complexes de manière visuelle et claire.



FIGURE 4.35 – Logo Draw io

- **EasyEDA**

EasyEDA est un outil en ligne dédié à la conception de circuits électroniques. Il permet de créer des schémas électriques, de concevoir des circuits imprimés (PCB), et même de

simuler certains comportements électroniques. Grâce à son interface accessible et ses bibliothèques de composants étendues,Il simplifie la transition entre la phase de conception et la fabrication d'un circuit, en intégrant des outils complets pour le schéma.



FIGURE 4.36 – Logo EasyEDA

4.3 Ressources matérielles

Dans cette partie, nous définirons les choix des composants de notre solution. Qui est divisé en parties, chacune ayant une fonction spécifique au sein du système.

4.3.1 Noeud de capteur

Cette partie est chargée de collecter et de transmettre de manière autonome les données sur la densité du trafic et intègre non seulement des capteurs mais également des composants de traitement des données sensorielles et une unité d'alimentation pour fournir l'énergie nécessaire au fonctionnement.

ESP-WROOM-32[9]

L'ESP-WROOM-32 est basé sur l'ESP32-D0WDQ6, qui fait partie de la série ESP32 et constitue une excellente solution pour les projets embarqués et IoT.
Il intègre le Wi-Fi, le Bluetooth et le Bluetooth Low Energy (BLE), garantissant une large gamme de communication avec divers appareils. Il se distingue par son processeur Xtensa LX6 dual-core 32 bits cadencé à 240 MHz, qui offre des performances élevées tout en maintenant une faible consommation d'énergie.

Spécification technique

- Mémoire Flash : Jusqu'à 16 Mo (souvent 4 Mo par défaut)
- SRAM : 520 Ko
- Connectivité : Wi-Fi 802.11 b/g/n, Bluetooth v4.2 BR/EDR et BLE
- GPIOs : Jusqu'à 34 broches GPIO
- Entrées analogiques : 18 canaux ADC 12 bits
- Sorties analogiques : 2 canaux DAC 8 bits
- Interfaces : SPI, I2C, I2S, UART, PWM, CAN, IR

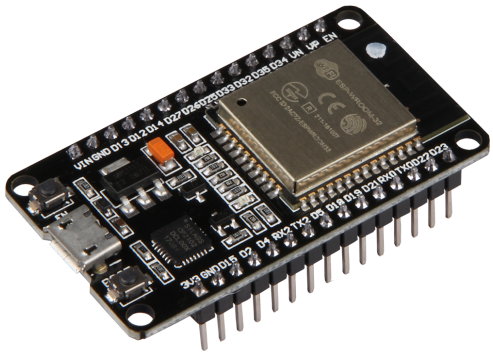


FIGURE 4.37 – ESP 32

Module de capteur VL53L0XV2[10]

Le VL53L0X est un capteur de la famille de la série ST VL5x et est développé par STMicroelectronics. C'est l'un des capteurs de distance qui s'appuient sur la technologie ToF (temps de vol), qui permet de mesurer la distance d'un objet en temps réel et avec une grande précision. Il existe plusieurs versions qui diffèrent en termes de portée de distance.
Ce qui en fait également l'un des meilleurs choix est sa grande capacité à détecter la distance, en plus de sa disponibilité et de sa facilité d'accès.

Principe de fonctionnenment

Le VL53L0X fonctionne sur le principe ToF (Time of Flight), où il émet des impulsions laser infrarouges et enregistre le temps de leur transmission. À l'arrivée, cette lumière est réfléchie par la cible, afin qu'elle puisse être captée par le récepteur, puis le temps de réception est enregistré. Grâce à ce processus, nous pouvons calculer la distance pour l'obtenir avec précision. La distance est calculée automatiquement à l'intérieur du capteur et nous l'obtenons ensuite en millimètres selon cette formule :
Distance réelle = ToF × c / 2

avec ToF = temps de vol de la lumière mesuré par le capteur (en secondes)

et

c = vitesse de la lumière (3 × 10 m/s)

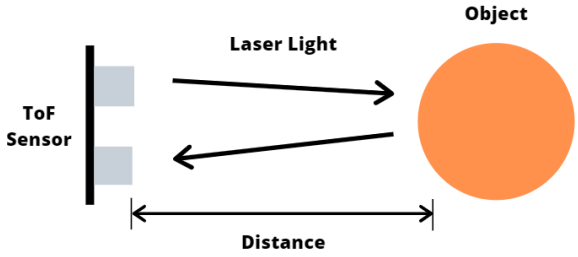


FIGURE 4.38 – Principe de fonctionnement de « Vl53l0X »

Spécifications techniques

- Gamme de mesure : 30 mm à 2000 mm dans des conditions optimales
- Précision : ±3%
- Résolution : 1 mm
- Tension de fonctionnement : 2.6V à 3.5V
- Interface de communication : I²C
- Classe laser : Classe 1 (inoffensif pour les yeux)
- Dimensions : 4.4 mm × 2.4 mm × 1 mm

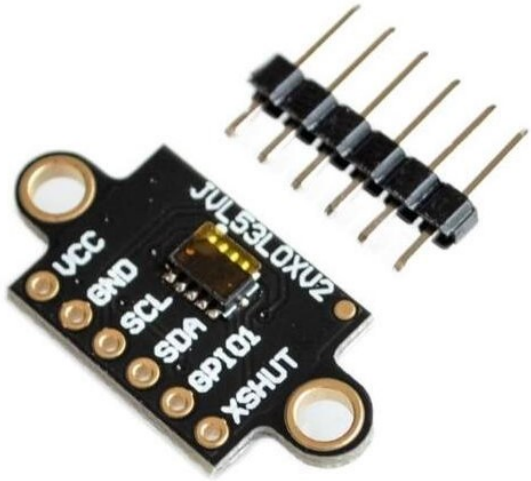


FIGURE 4.39 – VL53L0X

Capteur Ultrasons[11]

Un capteur à ultrasons est un dispositif permettant de mesurer la distance entre lui et un objet situé devant lui en utilisant des ondes sonores à haute fréquence. Il intègre à la fois un émetteur et un récepteur, qui régulent l'envoi et la réception des ondes, avec une portée maximale de 4 mètres.

Principe de fonctionnenment

Le capteur fonctionne en émettant une impulsion ultrasonore à l'aide de l'émetteur, se propageant à la vitesse du son dans l'air (environ 344 m/s). L'onde voyage dans l'air jusqu'à ce qu'elle rencontre un objet, qui la réfléchit vers le récepteur du capteur. Celui-ci mesure ensuite le temps écoulé entre l'émission et la réception de l'onde, puis calcule la distance en utilisant la vitesse du son (et non la vitesse de la lumière,
Cette équation est utilisée pour calculer la distance en convertissant le temps de vol de l'onde sonore en une distance exacte :

distance = (duration*0.0343)/2 ;

- Duration : Le temps mesuré qu'il faut à une onde sonore pour se déplacer vers un objet et revenir.

- 0,0343 : Vitesse du son dans l'air en cm/microseconde.

- /2 : Nous divisons par 2 pour obtenir la distance dans une direction, puisque le temps que nous obtenons reflète le temps du voyage aller et retour.

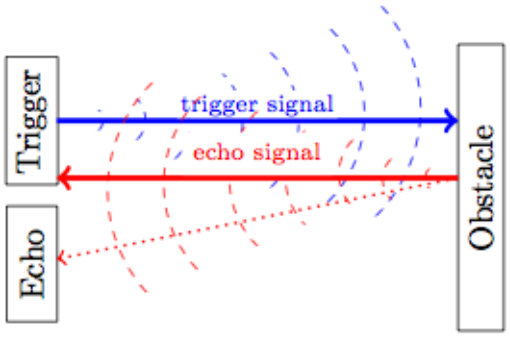


FIGURE 4.40 – Principe de fonctionnement du capteur d'ultrasons

Spécifications techniques

- Plage de mesure : 2 cm à 4 m
- Précision : ± 3 mm
- Fréquence d'oscillation : 40 kHz
- Alimentation : 5V
- Courant de fonctionnement : Environ 15 mA
- Temps de mesure : 50 ms pour chaque mesure
- Angle de détection : 15°
- Dimensions : 45 mm × 20 mm × 15 mm
- Poids : Environ 15 g



FIGURE 4.41 – Capteur ultrason HY-SRF05

4.3.2 Circuit d'alimentation

Le circuit d'alimentation est la partie chargée de maintenir le système en fonctionnement le plus longtemps possible en lui fournissant l'énergie nécessaire. Il se compose de deux éléments de base : la batterie et le chargeur.

Batterie Li-ion 2500mAh[12]

La batterie lithium-ion de 2 500 mAh est une source d'alimentation rechargeable légère et excellente pour les applications de systèmes embarqués tels que les microcontrôleurs. Nous avons choisi cette batterie pour sa grande capacité à fournir un temps de charge acceptable pour notre systéme, en calculant la consommation de chaque composant par rapport à ce que fournit la batterie :

- ESP32 (Mode Actif) :Pic courant (Wi-Fi activé) : Jusqu'à 240 mA.

- Capteur VL53L0X : 20 mA (en fonctionnement continu).

- Capteur HC-SR04 : 15 mA .

- Module de protection (BMS) : 5 mA.

- Consommation Totale : Jusqu'à 280 mA (pic).

- Durée d'Autonomie Estimée : 2500mA / 280 mA = 9h (environs)

Spécifications techniques

- Capacité : 2500 mAh
- Tension nominale : 3,7V
- Tension de charge : 4,2V (max)
- Courant de décharge maximal : Environ 10A (selon la qualité de la batterie)
- Courant de charge : Généralement entre 0,5A et 1A (pour prolonger la durée de vie de la batterie)
- Dimensions : 18,6 mm de diamètre, 65 mm de longueur
- Poids : Environ 45 g



FIGURE 4.42 – Batterie Li-ion 2500mAh 3.7V

Battery Shield – 2 × 18650[13]

Le Battery Shield 2×18650 est un module d'alimentation intelligent conçu pour les applications de projets embarqués qui nécessitent une source d'alimentation portable et fiable. Il intègre également deux batteries Li-ion 18650, ce qui double la puissance de charge par heure et prolonge la durée de fonctionnement du système. Il fournit également un système de sécurité et de charge complet, combinant Boost DC-DC, un régulateur 3,3 V (LDO) et un circuit de protection (BMS) à l'intérieur, éliminant ainsi le besoin de circuits externes. Il fournit des sorties 5V (USB)/(broche) et 3,3V (broche) pour couvrir tous les besoins.

Spécifications techniques

- Entrée : 3,7V à 4,2V (typique des batteries 18650)
- Sortie : 5V
- Protection intégrée : contre la surcharge, la décharge excessive et les courts-circuits
- Régulateur de tension : module abaisseur (Buck converter) pour une sortie 5V stable
- Efficacité : environ 90%



FIGURE 4.43 – Module de charge

4.3.3 Circuit de recharge

Panneau Solaire 4W/6V[14]
Le panneau solaire est une source d'énergie renouvelable idéale pour les projets intégrés, car il permet de recharger la batterie et de faire fonctionner directement des systèmes à faible consommation. Ce qui en fait le meilleur choix pour ce projet est sa petite taille, qui facilite son processus d'intégration et ses performances adaptées à ce système externe et indépendant.
Il est important de fournir les conditions idéales, car il est préférable que son angle soit de 90 degrés pour mieux capter la lumière du soleil et la convertir en énergie électrique.

Voici un bref aperçu du temps de recharge par rapport à la consommation.

Temps de Recharge d'une Batterie Li-ion :

Énergie Requise :
Capacité : 2500mAh $\times$ 3.7V = 9.25 Wh.

Pertes de charge :

9.25Wh/0.713.2Wh.

Estimation de temps de recharge :

13.2Wh/ 4W 3.3 heures

Spécifications techniques

- Puissance : 4W
- Tension : 6V
- Courant de sortie : 180 mA
- Dimensions : 125 mm × 135 mm



FIGURE 4.44 – Panneau Solaire 4W/6V

Module abaisseur et élévateur d'énergie solaire LM2596[15]

Le LM2596 est un circuit intégré optimisé pour la charge solaire des batteries lithium-ion, intégrant une technologie MPPT simplifiée permettant d'extraire le maximum d'énergie du panneau solaire quelles que soient les conditions d'ensoleillement. Ce dispositif nécessite un réglage précis de la tension pour assurer la sécurité des contrôleurs et capteurs connectés. Il incorpore plusieurs protections essentielles : blocage de la charge si la tension batterie excède 4,25V, coupure en cas de décharge profonde (<2,9V), et isolation de la batterie pour éviter toute décharge inverse vers le panneau solaire pendant les périodes nocturnes ou sans lumière.

Spécifications techniques

- Entrée : Tension d'entrée de 2,5V à 5V
- Sortie : 5V ou 12V
- Courant de sortie : Jusqu'à 1A
- Composant utilisé : Convertisseur DC-DC abaisseur ou élévateur (Buck ou Boost)
- Efficacité : Environ 85% ou plus

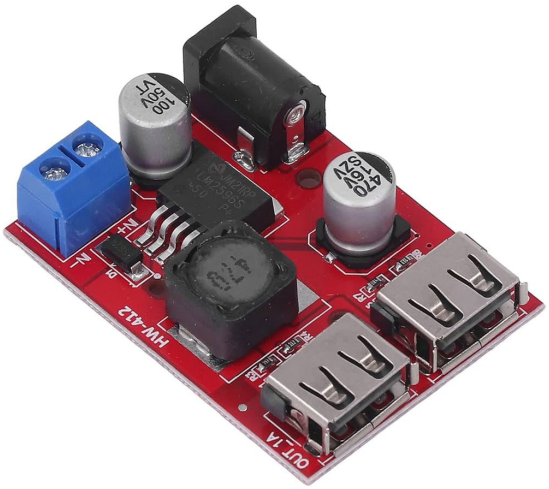


FIGURE 4.45 – Module LM2596 (Convertisseur DC-DC)

4.4 Circuit électrique

La figure 4.46 présente le schéma électrique du circuit, illustrant l'assemblage des différentes composantes utilisées et leurs interconnexions. Les principaux composants sont :
- Microcontrôleur ESP32 : Gère les données.
- Capteurs VL53L0X : Détectent les véhicules.
- Capteur à ultrasons : Mesure les distances.
- Système d'alimentation : Fournit l'énergie.

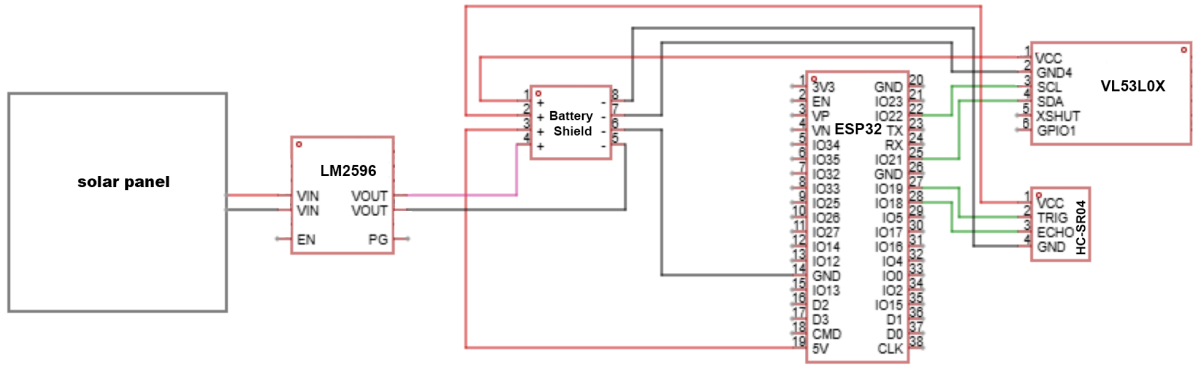


FIGURE 4.46 – Schéma électrique du circuit

Dans notre système de surveillance de trafic, l'alimentation constitue une contrainte majeure, car elle repose sur un panneau solaire et des batteries Li-ion de 2500 mAh. Utilisé en extérieur, le système doit assurer une alimentation stable pour les capteurs VL53L0X et l'ESP32, malgré les variations de production énergétique du panneau solaire dues aux conditions météorologiques (par exemple, un faible ensoleillement la nuit ou par temps nuageux). La consommation totale estimée du système étant relativement importante, l'autonomie offerte par une seule batterie est limitée. Pour garantir un fonctionnement continu, notamment en l'absence prolongée de lumière solaire, il est nécessaire d'envisager l'ajout de batteries supplémentaires ou l'intégration d'un système de recharge plus performant, afin de prolonger l'autonomie aussi longtemps que possible.

4.5 Architecture du système

La figure ci-dessous illustre l'architecture intégrée du système de surveillance de trafic, comprenant un front-end interactif pour l'utilisateur, un back-end basé sur Flask et SQLite pour le traitement des données, et des nœuds de capteurs alimentés par des panneaux solaires. Les capteurs, équipés de microcontrôleurs et de batteries, collectent les données en temps réel et les transmettent au back-end via MQTT, avec des possibilités d'amélioration pour optimiser la disposition des composants.

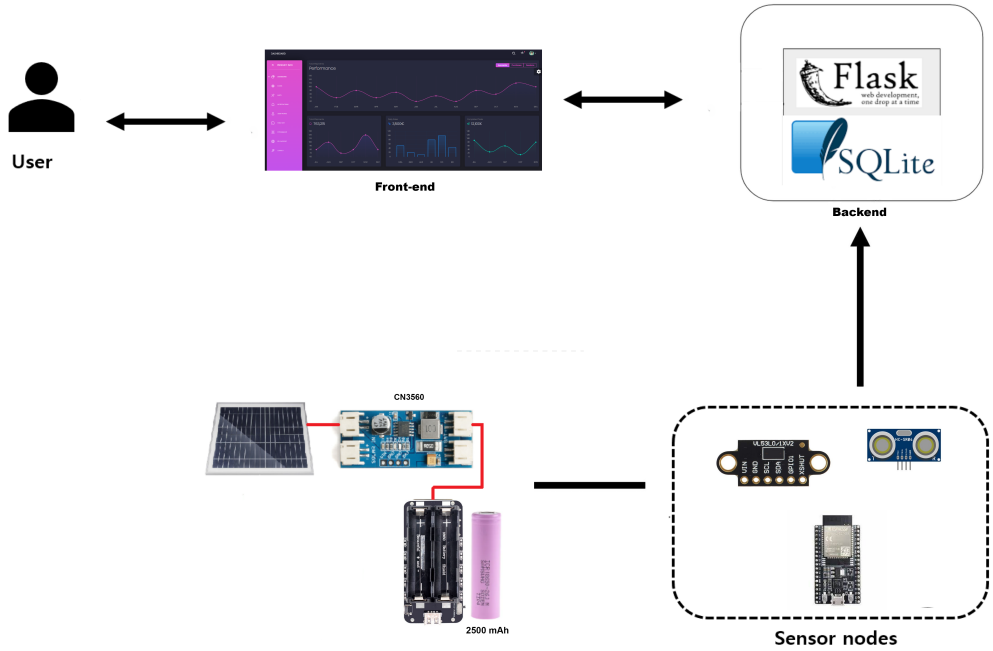


FIGURE 4.47 – Le schéma du système

4.6 Implémentation et interfaces

4.6.1 Configuration du capteur VL53L0X[16]

Le fonctionnement du capteur VL53L0X peut être ajusté à l'aide de plusieurs paramètres proposés par sa bibliothèque logicielle. Ces paramètres permettent de contrôler des aspects tels que le signal, les impulsions laser, le temps de mesure ou la fréquence I²C. Leur configuration est essentielle pour adapter le capteur aux exigences du système en matière de portée, de précision et de réactivité.

Dans notre application, visant à détecter efficacement le passage de véhicules, nous avons utilisé un ensemble de fonctions permettant d'optimiser le comportement du capteur dans un environnement dynamique.

- **Taux minimal de signal de retour : sensor.setSignalRateLimit()**

Cette fonction permet de définir une limite minimale du taux de signal de retour accepté par le capteur, exprimée en mégacounts par seconde (MCPS).
Elle agit comme un filtre afin de rejeter les mesures ayant un signal trop faible, souvent causé par des surfaces peu réfléchissantes ou trop éloignées.

L'objectif de ce paramètre est d'améliorer la fiabilité des mesures en excluant les données de mauvaise qualité. Typiquement, une valeur comme 0.1 MCPS est utilisée comme seuil minimal. Une valeur trop basse peut augmenter le risque de mesures erronées, tandis qu'une valeur trop élevée peut empêcher la détection d'objets faiblement réfléchissants.

◦ **Période d'impulsion VCSEL pour la plage de pré-détection : sensor.setVcselPulsePeriod(VL53L0X : :VcselPeriodPreRange, valeur)**

Cette fonction configure la durée d'impulsion du laser VCSEL (Vertical-Cavity Surface-Emitting Laser) durant la phase de pré-détection (Pre-Range), le Pre-Range correspond à une étape de calibration interne où le capteur optimise son comportement avant la mesure réelle.
Cette durée est exprimée en périodes d'horloge internes (unités arbitraires, souvent 12, 14, 16 ou 18). Une durée plus longue permet une meilleure sensibilité mais augmente le temps nécessaire à chaque mesure. À l'inverse, une durée plus courte favorise la vitesse d'acquisition au détriment de la précision, surtout à courte portée.

◦ **Période d'impulsion VCSEL pour la plage de détection finale : sensor.setVcselPulsePeriod(VL53L0X : :VcselPeriodFinalRange, valeur)**

Cette fonction ajuste la durée de l'impulsion VCSEL, mais pendant la phase finale de mesure (Final Range), c'est-à-dire lorsque la distance effective est mesurée une plus grande période permet d'augmenter la portée et la qualité de mesure dans des environnements difficiles (faible lumière, objets absorbants), au prix d'un temps de mesure plus long. Typiquement, les valeurs autorisées sont 8, 10, 12 ou 14. Ce paramètre a un impact direct sur la précision et la portée maximale du capteur.

◦ **Durée allouée à une mesure : sensor.setMeasurementTimingBudget(valeur)**

Ce paramètre définit le budget temporel alloué à une mesure complète, en microsecondes. Il englobe toutes les phases du cycle de mesure, y compris les calibrations internes. Plus le timing budget est élevé, plus le capteur peut accumuler d'informations et fournir une mesure précise et stable. Cependant, cela ralentit la cadence d'échantillonnage.
Des valeurs typiques vont de 20 000 µs (rapide, mais moins précis) à 200 000 µs (plus lent, mais plus fiable). Il est important de trouver un compromis entre la réactivité du système et la qualité des données de distance.

◦ **Démarrage des mesures continues avec un intervalle défini : sensor.startContinuous(valeur)**

Cette commande lance le mode de mesure continue du capteur. Elle peut être appelée avec un paramètre facultatif représentant l'intervalle minimal entre deux mesures (en millisecondes).
Ce mode est utile lorsque des mesures en temps réel sont requises, par exemple dans un système de suivi de mouvement ou de détection d'obstacles. Une fréquence trop élevée peut générer un traitement excessif ou des lectures bruitées si le timing budget est mal ajusté. À l'inverse, une fréquence trop basse réduit la réactivité du système.

◦ **Durée maximale d'attente avant déclenchement d'un dépassement de délai :**
sensor.setTimeout(valeur)

Cette fonction définit un temps maximal d'attente pour une mesure, en millisecondes. Si la lecture de distance ne se termine pas dans ce délai, elle est interrompue pour éviter que le programme ne reste bloqué indéfiniment.
Ce mécanisme est essentiel pour garantir la robustesse du système, surtout en cas de dysfonctionnement ou d'environnement défavorable. La valeur choisie doit être légèrement supérieure au timing budget de mesure pour permettre l'achèvement normal des opérations, sans générer de faux positifs.

◦ **Définition de la fréquence du bus I2C :**
Wire.setClock(valeur)

Cette commande ajuste la fréquence de communication I²C entre le microcontrôleur et le capteur VL53L0X, en hertz. Par défaut, cette fréquence est souvent fixée à 100 kHz, mais elle peut être augmentée jusqu'à 400 kHz pour améliorer la vitesse de transmission des données.
Cette optimisation est surtout utile lorsque plusieurs capteurs ou périphériques partagent le bus I²C. Toutefois, des fréquences trop élevées peuvent rendre la communication instable si les câbles sont longs ou les composants peu tolérants.

4.6.2 Test et résultats

Le capteur de distance VL53L0X offre plusieurs modes de configuration permettant d'adapter son fonctionnement selon les besoins spécifiques de l'application.
Deux modes principaux se distinguent : un mode privilégiant la fiabilité des mesures et un autre privilégiant la rapidité de détection. Chacun de ces modes utilise un jeu de paramètres spécifiques qui influencent la précision, la portée, la latence, et la robustesse face aux interférences.

4.3.2.1 Calibrage du capteur

Pour améliorer la précision des mesures, un calibrage manuel du capteur VL53L0X a été réalisé en plaçant un objet à une distance connue de 10 cm. Dix mesures successives ont été relevées, puis la moyenne a été calculée. L'écart moyen observé entre les mesures et la distance réelle a été d'environ 26 mm, ce qui a permis de définir un offset de -26 mm appliqué dans le code pour corriger les lectures. Ce type de calibrage est fortement conseillé afin de compenser les erreurs systématiques du capteur et garantir des mesures plus fiables en conditions réelles.

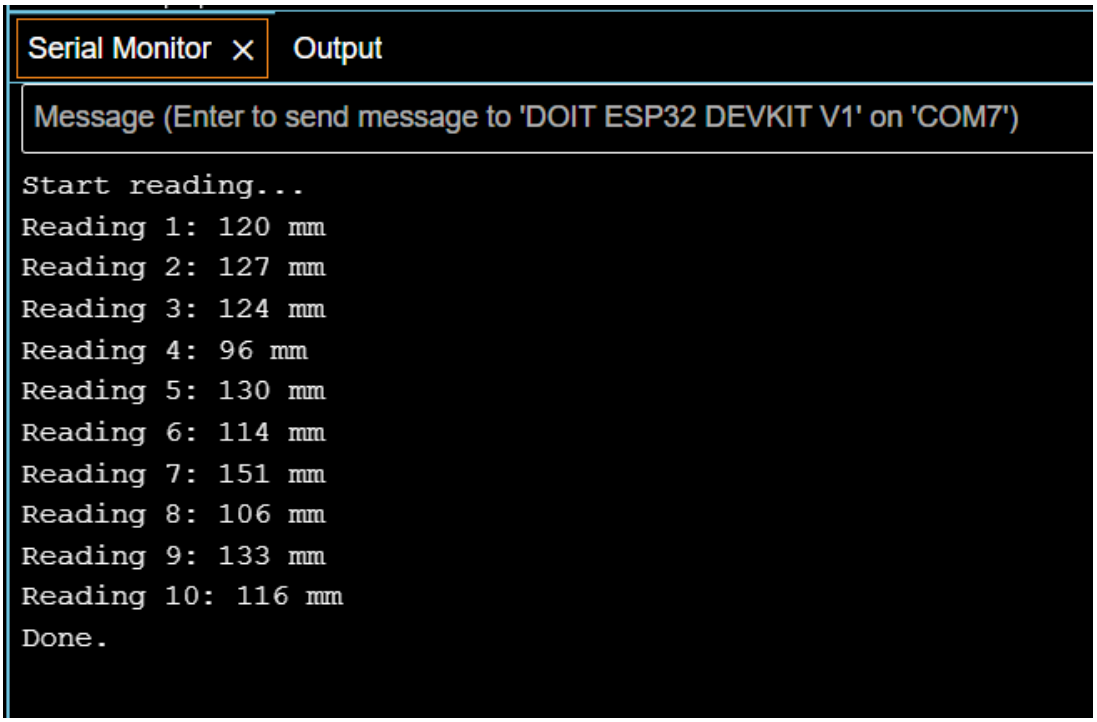


FIGURE 4.48 – Résultat de calibrage

Le calcul de l'offset est effectué en faisant la différence entre la distance réelle et la moyenne des mesures, et comme montré dans cet exemple, l'offset est égal à -21 mm :

120 + 127 + 124 + 96 + 130 + 114 + 151 + 106 + 133 + 116 = 1217

1217 / 10 = 121.7 mm

Offset = 100 mm - 121.7 mm = -21.7 mm

4.3.2.2 Test orienté vers la fiabilité

Dans ce mode, l'objectif principal est de privilégier la stabilité et la précision des mesures, même dans des environnements complexes ou perturbés.
Il s'agit d'assurer une détection fiable, notamment lorsque les objets à mesurer présentent une faible réflectivité ou sont exposés à des conditions lumineuses défavorables. Ce mode est particulièrement adapté aux applications où la rigueur des mesures est essentielle, même si cela implique un temps de réponse plus long.
Une explication des valeurs des paramètres choisis, accompagnée d'un exemple de résultats du test, est présentée dans cette partie :

- **sensor.setSignalRateLimit(0.25) :**

- Exige un signal plus fort pour valider une mesure.
-Réduit les erreurs dues à la faible réflectivité ou à la lumière parasite.

- **sensor.setMeasurementTimingBudget(50000) :**

- Un temps plus long permet au capteur d'émettre plusieurs impulsions et d'utiliser des moyennes internes.

- **sensor.setVcselPulsePeriod(VL53L0X : :VcselPeriodPreRange, 18) ;**

- **sensor.setVcselPulsePeriod(VL53L0X : :VcselPeriodFinalRange, 18) ;**

- Donne au signal plus de chance de rebondir sur des surfaces lointaines ou sombres
- Augmente la latence .

- **sensor.startContinuous(100) :**

-Démarre les mesures continues toutes les 100 ms.

- **sensor.setTimeout(1000) :**

- Donne le temps nécessaire pour les cas complexes (longue portée, faible signal).

- **Wire.setClock(100000) :**

- Stable, fiable et compatible avec tous les périphériques.
- Pas très rapide .

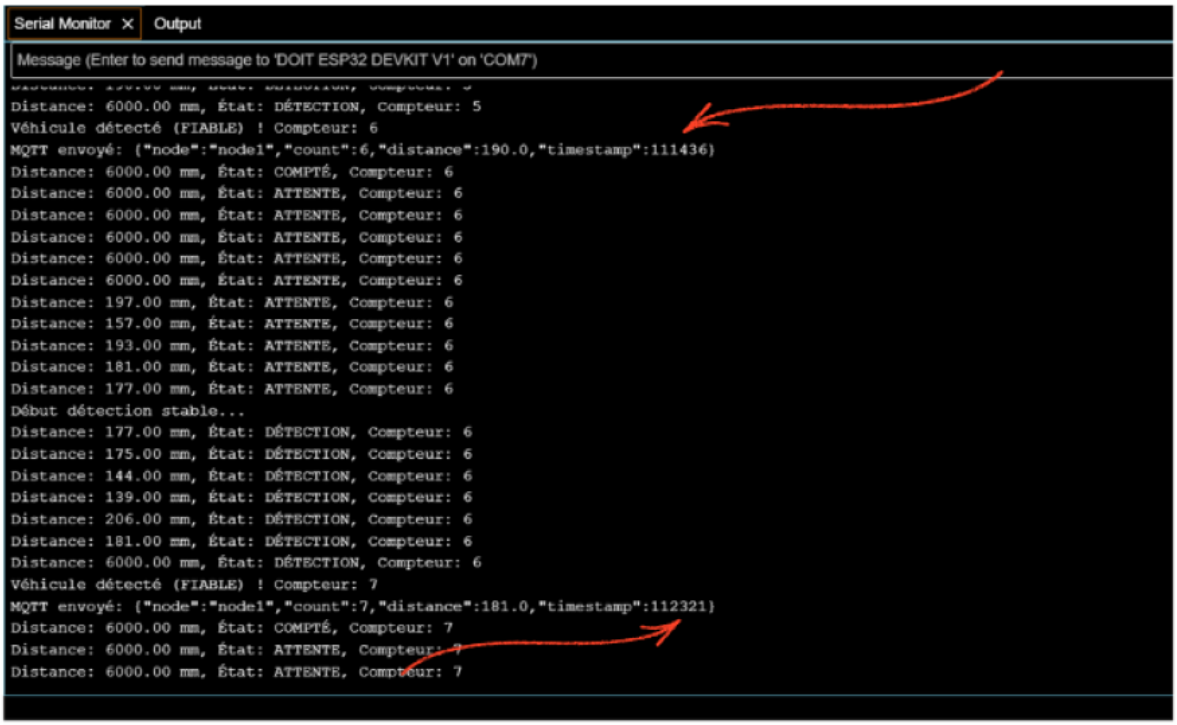


FIGURE 4.49 – Résultat du test orienté vers la fiabilité

Dans ce test, on constate que la détection et la mesure sont globalement stables, ce qui est un point positif.
Cependant, la portée du capteur reste réduite et la rapidité de détection est très lente. Cela signifie que le système n'est pas capable de détecter efficacement des véhicules circulant à grande vitesse sur la route.

4.3.2.3 Test orienté vers la rapidité

Ce mode est conçu pour des situations où la rapidité de détection est prioritaire, comme dans le cas de véhicules se déplaçant à grande vitesse. Les paramètres sont configurés pour réduire au maximum le temps entre deux mesures, permettant une réactivité élevée. Toutefois, cette optimisation de la vitesse se fait au détriment de la portée et de la précision, rendant les mesures plus sensibles au bruit, moins fiables sur les longues distances, et moins adaptées aux environnements complexes.

- **sensor.setSignalRateLimit(0.1) :**

- Permet des mesures même avec peu de signal.
- Risque d'accepter des mesures erronées si le bruit est élevé

- **sensor.setMeasurementTimingBudget(15000) :**

- Rapide mais utile pour les obstacles proches.
- Moins de temps pour compenser les erreurs ou bruit .

- **sensor.setVcselPulsePeriod(VL53L0X : :VcselPeriodPreRange, 8) ;**

- **sensor.setVcselPulsePeriod(VL53L0X : :VcselPeriodFinalRange, 8) ;**

- Lecture rapide.
- Diminue la portée et précision.

- **sensor.startContinuous(0) :**

- Lance des mesures le plus vite possible sans pause.

- **sensor.setTimeout(200) :**

- Temps d'attente réduit.
-Risque d'abandonner certaines mesures valides.

- **Wire.setClock(400000) :**

- Transfert de données plus rapide entre microcontrôleur et capteur.
- Moins stable sur longues distances.

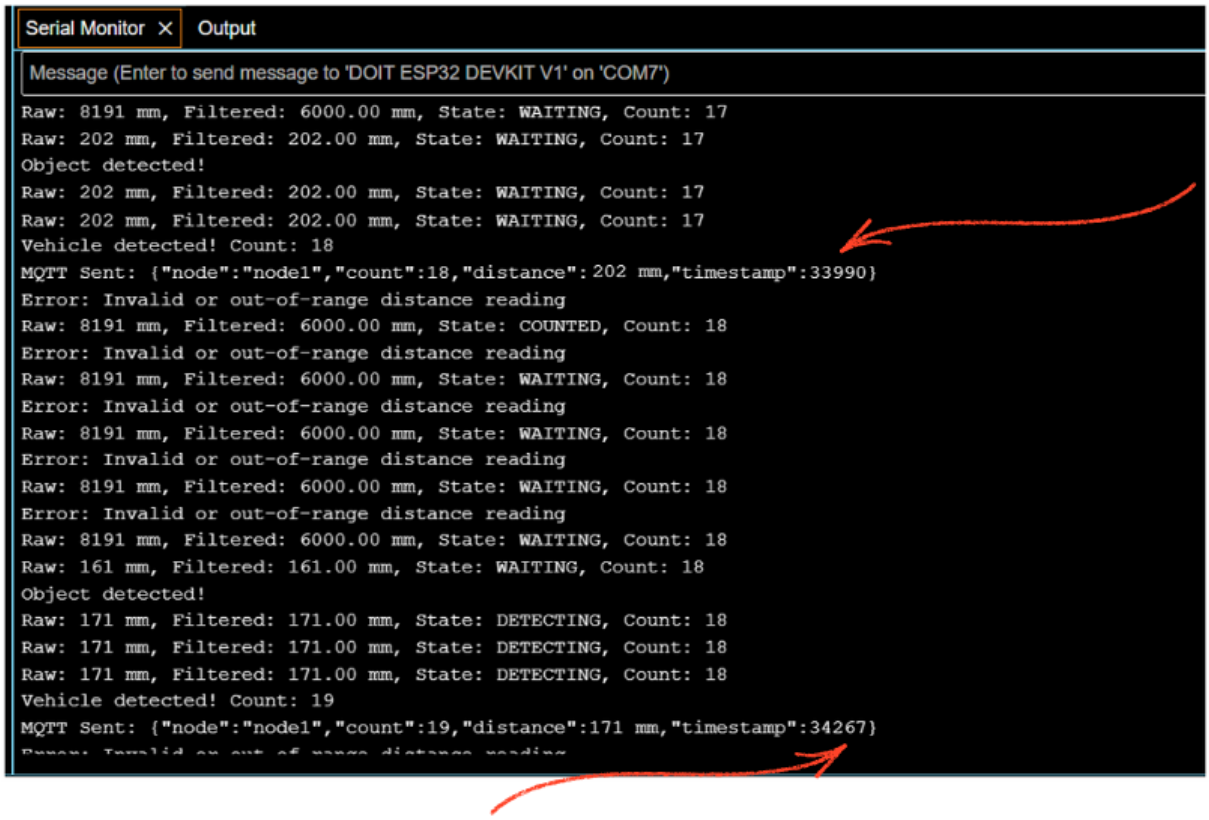


FIGURE 4.50 – Résultat du test orienté vers la rapidité

Dans ce test, on observe que le temps de réponse est très court, ce qui permet une détection plus rapide des véhicules.
Cependant, la modification des paramètres pour accélérer la prise de mesure a eu un impact sur la qualité des données. Les mesures deviennent moins fiables, parfois erronées, et le système se montre plus sensible aux interférences et aux bruits environnants.

4.6.3 Réalisation du système de détection

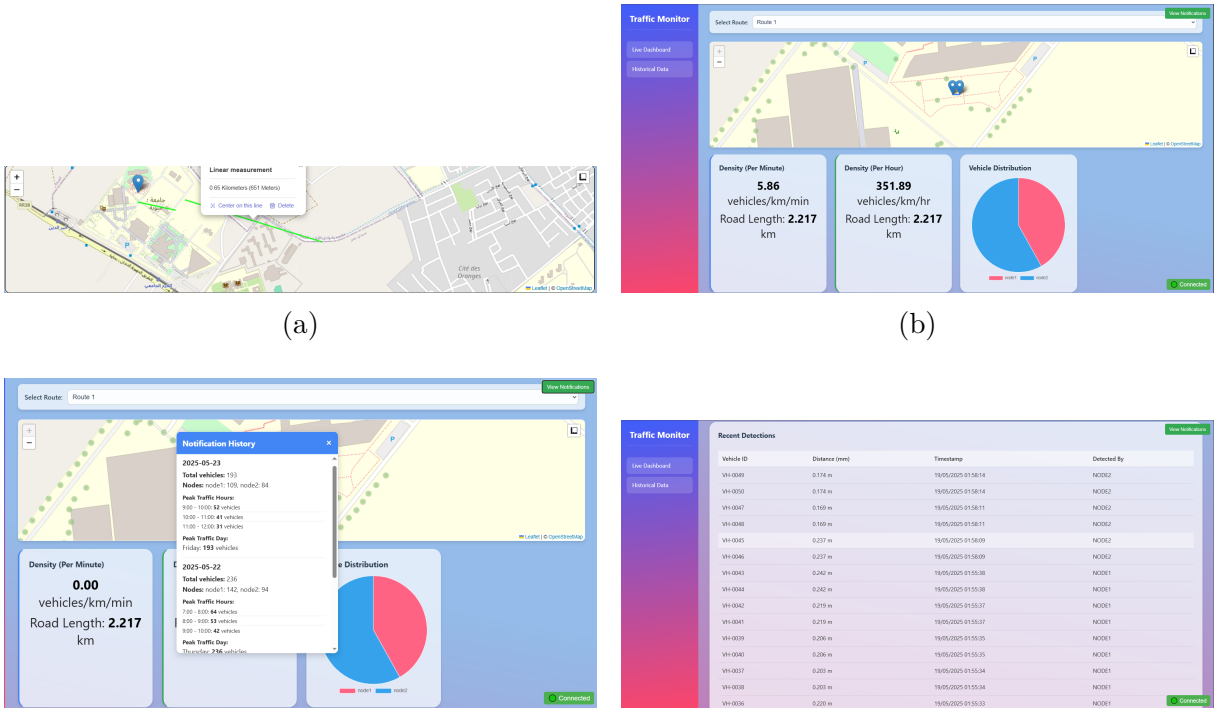
Les interfaces du tableau de bord

Les figures ci-dessous représentent l'interface de notre tableau de bord, conçu pour la visualisation et le suivi en temps réel des données collectées. Cette interface comprend une zone dédiée aux compteurs en temps réel, ainsi qu'une zone de graphiques permettant de visualiser l'évolution des données au fil du temps sous forme de courbes dynamiques et interactives. Afin de renforcer la clarté et l'interprétation visuelle des données, nous intégrons également une carte interactive permettant de localiser les détections et d'observer les trajets en temps réel.

Sur cette carte, nous utilisons le Leaflet Measurement Plugin pour mesurer la longueur des routes en sélectionnant deux points. Cette longueur (exprimée en kilomètres) est essentielle au calcul de la densité du trafic.

Le système enregistre les détections récentes. Ensuite, il compte les véhicules détectés pendant une période donnée , puis applique la formule suivante pour déterminer la densité :

Densité=Nombre de véhicules longueur de la route(Km)*Durée(minute ou heure)



(a) (b)

(c) (d)

FIGURE 4.51 – Interface de visualisation des données graphiques

Prototype final

Ces figures présentent le prototype final réel du système, dans lequel nous avons adapté l'utilisation de deux nœuds afin d'assurer une couverture complète de la route pour le co-voiturage. En effet, bien que la distance maximale théorique de détection des capteurs soit d'environ 2 mètres, l'intégration de deux nœuds permet de surmonter cette limitation et de garantir une détection fiable et continue sur l'ensemble de la zone surveillée.

Le montage est intégré à l'intérieur d'un boîtier conçu pour être autonome, grâce à un panneau solaire fixé au couvercle. Ce dernier alimente le système de manière durable et écologique. À l'intérieur, on retrouve deux batteries rechargeables assurant une alimentation continue.

Chaque nœud utilise un ESP32 comme microcontrôleur. Le premier nœud est équipé d'un capteur VL53L0X et d'un capteur à ultrasons, tandis que le deuxième nœud utilise uniquement un VL53L0X.



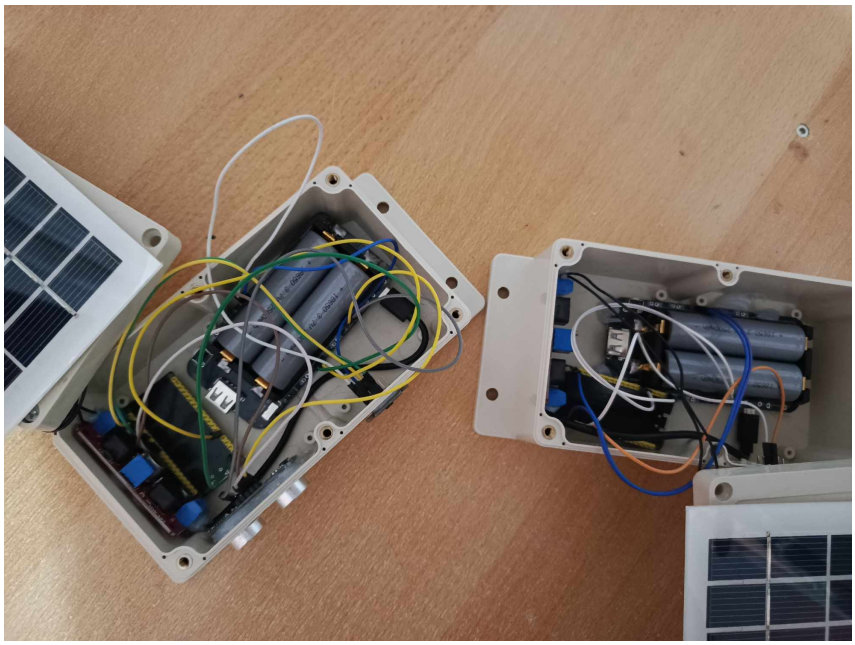


FIGURE 4.52 – Prototype final réel

4.7 Conclusion

Dans ce quatrième chapitre, nous avons abordé la phase de mise en œuvre concrète de notre système de détection de véhicules. Nous avons tout d'abord présenté l'environnement logiciel et matériel utilisé, ainsi que les bibliothèques et outils nécessaires au bon développement du projet. Par la suite, nous avons décrit les différentes étapes de la réalisation du système en mettant en évidence les contraintes rencontrées lors de l'exécution des tests.

Conclusion Générale

Au terme de ce rapport, nous avons détaillé la conception et la mise en œuvre d'un système de détection de véhicules reposant sur l'utilisation de capteurs de distance.
L'objectif principal de ce projet est de permettre une estimation fiable de la densité de trafic routier, tout en proposant une solution à la fois économique, pratique et facilement déployable sur différents types d'axes routiers.
L'intégration d'un tableau de bord interactif représente un atout majeur, car elle offre une visualisation claire et en temps réel des données collectées, facilitant ainsi l'analyse et la prise de décision.

Cette solution constitue une alternative aux méthodes classiques de surveillance de la circulation, souvent coûteuses et complexes à mettre en œuvre.
Ce système se distingue par sa simplicité d'installation, la précision de ses mesures, ainsi que par son faible coût, ce qui le rend accessible à grande échelle.

Par ailleurs, plusieurs pistes d'amélioration peuvent être envisagées afin d'augmenter l'efficacité et l'adaptabilité du système. L'une des évolutions majeures consisterait à intégrer une passerelle de communication pour optimiser la transmission des données, notamment sur de longues distances, et améliorer la couverture réseau.
De plus, l'intégration de différents types de capteurs pour renforcer la précision et la fiabilité du système, en particulier dans des environnements complexes ou soumis à des contraintes spécifiques.

Bibliographie

[1] https://inrix.com/scorecard/. heures.

[10] https://www.esp32learning.com/code/vl53l0x-time-of-flight-sensor-and-esp32.php. VL53L0XV2.

[11] https://cothings.net/products/hc-sr05hy-srf05-module-capteur-ultrasons-5-pines?_pos=3&_sid=0298c2277&_ss=r. Capteur Ultrasons.

[12] https://www.didactico.tn/produit/pile-li-ion-18650-rechargeable-2500mah. Batterie Li-ion 2500mAh.

[13] https://quartzcomponents.com/products/2-x-18650-battery-shield-module-for-arduino-esp8266-and-raspberry-pi. Battery Shield – 2 × 18650.

[14] https://solars-china.com/fr/Solaires/165165P-solar-panel.html. Panneau Solaire.

[15] https://www.ti.com/lit/ds/symlink/lm2596.pdf. LM2596.

[16] https://www.st.com/resource/en/user_manual/um2039-world-smallest-timeofflight-ranging-and-gesture-detection-sensor-application-programming-interface-stmicroelectronics.pdf. Paramètres VL53L0XV2.

[17] https://flask.palletsprojects.com/en/stable/. Flask.

[18] https://datatracker.ietf.org/doc/html/rfc6455. WebSocket.

[2] https://transport.ec.europa.eu/index_en. coût.

[3] https://www.smartmicro.com/fileadmin/media/Downloads/Automotive_Radar/Sensor_Data_Sheets_76-81GHz/UMRR_Automotive_Type_153_Data_Sheet.pdf. Smartmicro UMRR – 96.

[4] https://www.aldridgetrafficcontrollers.com.au/products/video-detection/traficam. FLIR TrafiCam x-stream™.

[6] https://www.traffictechnologytoday.com/. coût.

[7] https://www.traffictechnologytoday.com/. MQTT.

[8] https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. Pull-up resistor.

[9] https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. ESP-WROOM-32.